Quiniela

Repositorio de documentacion tecnica

Documentacion Maestra

Repositorio Quiniela

Version: 0.1.0
Estado: Borrador operativo
Fecha: 4 de abril de 2026

Documento interno de trabajo

Índice general

1 Mapa documental **13**
1.1 Descripcion . . . 13
1.2 Alcance . . . 13
1.3 Definiciones . . . 13
1.4 Reglas . . . 13
1.5 Edge Cases . . . 13
1.6 Invariantes . . . 14
1.7 Preguntas abiertas . . . 14

2 Vision y alcance **14**
2.1 Descripcion . . . 14
2.2 Alcance . . . 14
2.3 Definiciones . . . 14
2.4 Reglas . . . 14
2.5 Edge Cases . . . 15
2.6 Invariantes . . . 15
2.7 Preguntas abiertas . . . 15

3 Principios de ingenieria y arquitectura **15**
3.1 Descripcion . . . 15
3.2 Alcance . . . 15
3.3 Definiciones . . . 15
3.4 Reglas . . . 16
3.5 Edge Cases . . . 17
3.6 Invariantes . . . 17
3.7 Preguntas abiertas . . . 17

4 Glosario **17**
4.1 Descripcion . . . 17
4.2 Alcance . . . 18
4.3 Definiciones . . . 18
4.4 Reglas . . . 18
4.5 Edge Cases . . . 18
4.6 Invariantes . . . 18
4.7 Preguntas abiertas . . . 18

5 Fuentes y gobernanza **18**
5.1 Descripcion . . . 18
5.2 Alcance . . . 19
5.3 Definiciones . . . 19
5.4 Reglas . . . 19
5.5 Edge Cases . . . 20
5.6 Invariantes . . . 20
5.7 Preguntas abiertas . . . 20

6 Reglamento v1 **20**

6.1 Descripcion . . . 20
6.2 Alcance . . . 20
6.3 Definiciones . . . 20
6.4 Reglas . . . 21
6.4.1 Participacion . . . 21
6.4.2 Picks por partido . . . 21
6.4.3 Resultado oficial del partido . . . 21
6.4.4 Scoring . . . 21
6.4.5 Leaderboard y visibilidad . . . 22
6.4.6 Cierre del torneo . . . 22
6.5 Edge Cases . . . 22
6.6 Invariantes . . . 22
6.7 Preguntas abiertas . . . 22
6.8 Decisiones pendientes . . . 23

7 Estados y flujos **23**
7.1 Descripcion . . . 23
7.2 Alcance . . . 23
7.3 Definiciones . . . 23
7.4 Reglas . . . 23
7.5 Edge Cases . . . 24
7.6 Invariantes . . . 24
7.7 Preguntas abiertas . . . 25
7.8 Decisiones pendientes . . . 25

8 Picks y locks **25**
8.1 Descripcion . . . 25
8.2 Alcance . . . 25
8.3 Definiciones . . . 25
8.4 Reglas . . . 25
8.4.1 Reglas generales . . . 25
8.4.2 Picks por partido . . . 26
8.4.3 Predicciones especiales . . . 26
8.4.4 Visibilidad . . . 26
8.5 Edge Cases . . . 26
8.6 Invariantes . . . 26
8.7 Preguntas abiertas . . . 27
8.8 Decisiones pendientes . . . 27

9 Scoring y desempates **27**
9.1 Descripcion . . . 27
9.2 Alcance . . . 27
9.3 Definiciones . . . 27
9.4 Reglas . . . 27
9.4.1 Puntaje por partido . . . 27
9.4.2 Multiplicadores por fase . . . 28
9.4.3 Predicciones especiales . . . 28
9.4.4 Leaderboard . . . 28

9.4.5 Desempates . . . 28
9.5 Edge Cases . . . 29
9.6 Invariantes . . . 29
9.7 Preguntas abiertas . . . 29
9.8 Decisiones pendientes . . . 29

10 Edge Cases **29**
10.1 Descripcion . . . 29
10.2 Alcance . . . 29
10.3 Definiciones . . . 30
10.4 Reglas . . . 30
10.4.1 Partidos postergados . . . 30
10.4.2 Partidos suspendidos . . . 30
10.4.3 Partidos cancelados . . . 30
10.4.4 Resultados corregidos . . . 30
10.4.5 Comunicacion funcional al usuario . . . 31
10.5 Edge Cases . . . 31
10.6 Invariantes . . . 31
10.7 Preguntas abiertas . . . 31
10.8 Decisiones pendientes . . . 31

11 Vision arquitectonica **31**
11.1 Descripcion . . . 32
11.2 Alcance . . . 32
11.3 Definiciones . . . 32
11.4 Reglas . . . 32
11.4.1 Vista logica de ejecucion . . . 32
11.4.2 Tree conceptual de repositorios . . . 33
11.5 Edge Cases . . . 33
11.6 Invariantes . . . 34
11.7 Preguntas abiertas . . . 34

12 Arquitectura frontend **34**
12.1 Descripcion . . . 34
12.2 Alcance . . . 34
12.3 Definiciones . . . 34
12.4 Reglas . . . 34
12.4.1 Capas funcionales esperadas . . . 35
12.4.2 Tree arquitectonico sugerido . . . 35
12.4.3 Responsabilidades por bloque . . . 35
12.4.4 Criterios de uso de shared . . . 36
12.5 Edge Cases . . . 36
12.6 Invariantes . . . 36
12.7 Preguntas abiertas . . . 36
12.8 Decisiones pendientes . . . 36

13 Arquitectura backend **36**
13.1 Descripcion . . . 36

13.2 Alcance . . . 36
13.3 Definiciones . . . 37
13.4 Reglas . . . 37
13.4.1 Modulos de dominio esperados . . . 37
13.4.2 Capas internas esperadas . . . 37
13.4.3 Tree arquitectonico sugerido . . . 37
13.4.4 Servicios transversales esperados . . . 38
13.4.5 Limites de responsabilidad . . . 38
13.5 Edge Cases . . . 39
13.6 Invariantes . . . 39
13.7 Preguntas abiertas . . . 39
13.8 Decisiones pendientes . . . 39

14 Contratos y limites **39**
14.1 Descripcion . . . 39
14.2 Alcance . . . 39
14.3 Definiciones . . . 39
14.4 Reglas . . . 40
14.5 Edge Cases . . . 41
14.6 Invariantes . . . 41
14.7 Preguntas abiertas . . . 42
14.8 Decisiones pendientes . . . 42

15 Modulos y responsabilidades **42**
15.1 Descripcion . . . 42
15.2 Alcance . . . 42
15.3 Definiciones . . . 42
15.4 Reglas . . . 42
15.4.1 Tree modular esperado . . . 42
15.4.2 Responsabilidades por modulo . . . 43
15.4.3 Reglas de colaboracion entre modulos . . . 43
15.5 Edge Cases . . . 44
15.6 Invariantes . . . 44
15.7 Preguntas abiertas . . . 44
15.8 Decisiones pendientes . . . 44

16 Procesos de negocio **44**
16.1 Descripcion . . . 44
16.2 Alcance . . . 44
16.3 Definiciones . . . 45
16.4 Reglas . . . 45
16.4.1 Proceso de activacion por comprobante . . . 45
16.4.2 Proceso de captura y lock de picks . . . 45
16.4.3 Proceso de sincronizacion de resultados . . . 46
16.4.4 Proceso de scoring por partido . . . 46
16.4.5 Proceso de recalculo de ranking . . . 47
16.4.6 Proceso de archivado del torneo . . . 47
16.5 Edge Cases . . . 48

16.6 Invariantes . . . 48
16.7 Preguntas abiertas . . . 48
16.8 Decisiones pendientes . . . 48

17 Validacion y errores **48**
17.1 Descripcion . . . 48
17.2 Alcance . . . 48
17.3 Definiciones . . . 48
17.4 Reglas . . . 49
17.4.1 Validacion de frontera . . . 49
17.4.2 Taxonomia de errores . . . 49
17.4.3 Idempotencia . . . 50
17.4.4 Relacion entre error, log y auditoria . . . 50
17.5 Edge Cases . . . 50
17.6 Invariantes . . . 51
17.7 Preguntas abiertas . . . 51
17.8 Decisiones pendientes . . . 51

18 Endpoints conceptuales **51**
18.1 Descripcion . . . 51
18.2 Alcance . . . 51
18.3 Definiciones . . . 51
18.4 Reglas . . . 51
18.4.1 Operaciones publicas y autenticadas esperadas . . . 52
18.4.2 Operaciones administrativas esperadas . . . 52
18.4.3 Reglas de contrato conceptual . . . 53
18.5 Edge Cases . . . 53
18.6 Invariantes . . . 53
18.7 Preguntas abiertas . . . 53
18.8 Decisiones pendientes . . . 53

19 Estructura por features **54**
19.1 Descripcion . . . 54
19.2 Alcance . . . 54
19.3 Definiciones . . . 54
19.4 Reglas . . . 54
19.4.1 Features objetivo de v1 . . . 54
19.4.2 Tree funcional sugerido . . . 55
19.4.3 Responsabilidad por bloque . . . 55
19.4.4 Reglas de ownership por feature . . . 56
19.4.5 Reglas para shared . . . 56
19.4.6 Reglas de dependencia . . . 56
19.5 Edge Cases . . . 56
19.6 Invariantes . . . 57
19.7 Preguntas abiertas . . . 57
19.8 Decisiones pendientes . . . 57

20 Flujos y paginas **57**

20.1 Descripcion . . . 57
20.2 Alcance . . . 57
20.3 Definiciones . . . 57
20.4 Reglas . . . 58
20.4.1 Inventario funcional de paginas esperadas . . . 58
20.4.2 Flujos principales de participante . . . 58
20.4.3 Flujos principales de administracion . . . 59
20.4.4 Estados visibles minimos por pagina . . . 60
20.5 Edge Cases . . . 60
20.6 Invariantes . . . 60
20.7 Preguntas abiertas . . . 60
20.8 Decisiones pendientes . . . 60

21 Estado, validacion y datos **61**
21.1 Descripcion . . . 61
21.2 Alcance . . . 61
21.3 Definiciones . . . 61
21.4 Reglas . . . 61
21.4.1 Manejo de estado remoto . . . 61
21.4.2 Manejo de estado local . . . 62
21.4.3 Reglas de validacion . . . 62
21.4.4 Contratos y acceso a datos . . . 62
21.4.5 Datos minimos por feature . . . 62
21.4.6 Reglas de error y feedback . . . 63
21.5 Edge Cases . . . 63
21.6 Invariantes . . . 63
21.7 Preguntas abiertas . . . 64
21.8 Decisiones pendientes . . . 64

22 Modelo de dominio **64**
22.1 Descripcion . . . 64
22.2 Alcance . . . 64
22.3 Definiciones . . . 64
22.4 Reglas . . . 64
22.5 Edge Cases . . . 65
22.6 Invariantes . . . 65
22.7 Preguntas abiertas . . . 66

23 Entidades y relaciones **66**
23.1 Descripcion . . . 66
23.2 Alcance . . . 66
23.3 Definiciones . . . 66
23.4 Reglas . . . 66
23.5 Edge Cases . . . 67
23.6 Invariantes . . . 67
23.7 Preguntas abiertas . . . 67

24 Invariantes de datos **67**

24.1 Descripcion . . . 68
24.2 Alcance . . . 68
24.3 Definiciones . . . 68
24.4 Reglas . . . 68
24.5 Edge Cases . . . 68
24.6 Invariantes . . . 68
24.7 Preguntas abiertas . . . 69
24.8 Decisiones pendientes . . . 69

25 Auditoria y trazabilidad **69**
25.1 Descripcion . . . 69
25.2 Alcance . . . 69
25.3 Definiciones . . . 69
25.4 Reglas . . . 69
25.5 Edge Cases . . . 70
25.6 Invariantes . . . 70
25.7 Preguntas abiertas . . . 70
25.8 Decisiones pendientes . . . 70

26 API de resultados **70**
26.1 Descripcion . . . 70
26.2 Alcance . . . 70
26.3 Definiciones . . . 71
26.4 Reglas . . . 71
26.4.1 Datos minimos esperados del proveedor . . . 71
26.4.2 Adaptacion al dominio interno . . . 71
26.4.3 Criterios de aceptacion de un resultado externo . . . 71
26.4.4 Superficies internas consumidoras . . . 72
26.5 Edge Cases . . . 72
26.6 Invariantes . . . 72
26.7 Preguntas abiertas . . . 72
26.8 Decisiones pendientes . . . 72

27 Sincronizacion y fallback manual **72**
27.1 Descripcion . . . 73
27.2 Alcance . . . 73
27.3 Definiciones . . . 73
27.4 Reglas . . . 73
27.4.1 Secuencia funcional de sincronizacion automatica . . . 73
27.4.2 Uso del fallback manual . . . 73
27.4.3 Eventos que deben habilitar recalculo . . . 74
27.4.4 Politica de prioridad de fuentes . . . 74
27.4.5 Ventanas funcionales de sincronizacion . . . 74
27.5 Edge Cases . . . 74
27.6 Invariantes . . . 74
27.7 Preguntas abiertas . . . 75
27.8 Decisiones pendientes . . . 75

28 Email y notificaciones **75**
28.1 Descripcion . . . 75
28.2 Alcance . . . 75
28.3 Definiciones . . . 75
28.4 Reglas . . . 75
28.4.1 Correos minimos esperados en v1 . . . 76
28.4.2 Notificaciones visibles dentro del producto . . . 76
28.4.3 Reglas de disparo . . . 76
28.4.4 Mensajes que no son prioridad para v1 . . . 76
28.5 Edge Cases . . . 77
28.6 Invariantes . . . 77
28.7 Preguntas abiertas . . . 77
28.8 Decisiones pendientes . . . 77

29 Roles y permisos **77**
29.1 Descripcion . . . 77
29.2 Alcance . . . 77
29.3 Definiciones . . . 78
29.4 Reglas . . . 78
29.4.1 Roles funcionales minimos . . . 78
29.4.2 Matriz funcional de permisos . . . 78
29.4.3 Reglas de interpretacion . . . 79
29.5 Edge Cases . . . 79
29.6 Invariantes . . . 79
29.7 Preguntas abiertas . . . 79
29.8 Decisiones pendientes . . . 79

30 Seguridad aplicativa **79**
30.1 Descripcion . . . 80
30.2 Alcance . . . 80
30.3 Definiciones . . . 80
30.4 Reglas . . . 80
30.4.1 Controles minimos esperados . . . 80
30.4.2 Autenticacion y autorizacion . . . 80
30.4.3 Validacion y manejo de entrada . . . 81
30.4.4 Configuracion y secretos . . . 81
30.4.5 Manejo de errores . . . 81
30.5 Edge Cases . . . 81
30.6 Invariantes . . . 81
30.7 Preguntas abiertas . . . 82
30.8 Decisiones pendientes . . . 82

31 Logging, observabilidad y auditoria **82**
31.1 Descripcion . . . 82
31.2 Alcance . . . 82
31.3 Definiciones . . . 82
31.4 Reglas . . . 82
31.4.1 Responsabilidad de cada capa . . . 83

31.4.2 Eventos funcionales que deben auditarse . . . 83
31.4.3 Campos minimos esperados . . . 83
31.4.4 Correlacion esperada . . . 84
31.5 Edge Cases . . . 84
31.6 Invariantes . . . 84
31.7 Preguntas abiertas . . . 84
31.8 Decisiones pendientes . . . 84

32 Contingencia de resultados **84**
32.1 Descripcion . . . 84
32.2 Alcance . . . 85
32.3 Definiciones . . . 85
32.4 Reglas . . . 85
32.4.1 Eventos que pueden activar contingencia . . . 85
32.4.2 Secuencia minima de contingencia . . . 85
32.4.3 Responsabilidad operativa minima . . . 85
32.4.4 Comunicacion funcional . . . 86
32.5 Edge Cases . . . 86
32.6 Invariantes . . . 86
32.7 Preguntas abiertas . . . 86
32.8 Decisiones pendientes . . . 86

33 Estrategia de testing **86**
33.1 Descripcion . . . 86
33.2 Alcance . . . 87
33.3 Definiciones . . . 87
33.4 Reglas . . . 87
33.4.1 Prioridades de riesgo . . . 87
33.4.2 Distribucion recomendada por nivel . . . 87
33.4.3 Cobertura esperada por area . . . 88
33.4.4 Herramientas base aprobadas . . . 88
33.5 Edge Cases . . . 88
33.6 Invariantes . . . 88
33.7 Preguntas abiertas . . . 88
33.8 Decisiones pendientes . . . 88

34 Casos criticos de prueba **89**
34.1 Descripcion . . . 89
34.2 Alcance . . . 89
34.3 Definiciones . . . 89
34.4 Reglas . . . 89
34.4.1 Inventario minimo de casos criticos . . . 89
34.4.2 Casos criticos adicionales recomendados . . . 90
34.4.3 Asignacion por nivel de prueba . . . 90
34.5 Edge Cases . . . 90
34.6 Invariantes . . . 90
34.7 Preguntas abiertas . . . 91
34.8 Decisiones pendientes . . . 91

35 Criterios de aceptacion **91**
35.1 Descripcion . . . 91
35.2 Alcance . . . 91
35.3 Definiciones . . . 91
35.4 Reglas . . . 91
35.4.1 Criterios minimos por flujo . . . 92
35.4.2 Criterios de UAT minimo . . . 92
35.5 Edge Cases . . . 93
35.6 Invariantes . . . 93
35.7 Preguntas abiertas . . . 93
35.8 Decisiones pendientes . . . 93

36 Principios UI UX **93**
36.1 Descripcion . . . 93
36.2 Alcance . . . 93
36.3 Definiciones . . . 93
36.4 Reglas . . . 94
36.4.1 Principios de experiencia para participante . . . 94
36.4.2 Principios de experiencia para administracion . . . 94
36.4.3 Patrones funcionales esperados . . . 94
36.4.4 Friccion aceptable . . . 94
36.5 Edge Cases . . . 95
36.6 Invariantes . . . 95
36.7 Preguntas abiertas . . . 95
36.8 Decisiones pendientes . . . 95

37 Pantallas y estados **95**
37.1 Descripcion . . . 95
37.2 Alcance . . . 95
37.3 Definiciones . . . 95
37.4 Reglas . . . 96
37.4.1 Inventario minimo de pantallas . . . 96
37.4.2 Estados visibles especialmente sensibles . . . 96
37.4.3 Reglas para estados de error . . . 97
37.4.4 Reglas para estados de carga y vacio . . . 97
37.5 Edge Cases . . . 97
37.6 Invariantes . . . 97
37.7 Preguntas abiertas . . . 97
37.8 Decisiones pendientes . . . 97

38 Accesibilidad y microcopy **97**
38.1 Descripcion . . . 98
38.2 Alcance . . . 98
38.3 Definiciones . . . 98
38.4 Reglas . . . 98
38.4.1 Principios de microcopy . . . 98
38.4.2 Mensajes funcionales prioritarios . . . 98
38.4.3 Lineamientos de accesibilidad minima . . . 99

38.4.4 Tono recomendado . . . 99
38.4.5 Mensajes que deben evitarse . . . 99
38.5 Edge Cases . . . 99
38.6 Invariantes . . . 99
38.7 Preguntas abiertas . . . 99
38.8 Decisiones pendientes . . . 100

39 Roadmap y fases **100**
39.1 Descripcion . . . 100
39.2 Alcance . . . 100
39.3 Definiciones . . . 100
39.4 Reglas . . . 100
39.4.1 Fases del roadmap documental . . . 100
39.4.2 Estado actual del roadmap . . . 101
39.4.3 Siguiente tramo recomendado . . . 101
39.5 Edge Cases . . . 101
39.6 Invariantes . . . 101
39.7 Preguntas abiertas . . . 102
39.8 Decisiones pendientes . . . 102

40 Backlog documental **102**
40.1 Descripcion . . . 102
40.2 Alcance . . . 102
40.3 Definiciones . . . 102
40.4 Reglas . . . 102
40.4.1 Backlog activo por prioridad . . . 102
40.4.2 Tipos de trabajo admitidos en backlog . . . 103
40.5 Edge Cases . . . 103
40.6 Invariantes . . . 103
40.7 Preguntas abiertas . . . 103
40.8 Decisiones pendientes . . . 104

41 Decisiones pendientes **104**
41.1 Descripcion . . . 104
41.2 Alcance . . . 104
41.3 Definiciones . . . 104
41.4 Reglas . . . 104
41.5 Registro activo . . . 104
41.5.1 Prioridad alta . . . 104
41.5.2 Prioridad media . . . 105
41.5.3 Prioridad baja . . . 105
41.6 Edge Cases . . . 106
41.7 Invariantes . . . 106
41.8 Preguntas abiertas . . . 106

42 Riesgos y mitigaciones **106**
42.1 Descripcion . . . 106
42.2 Alcance . . . 106

42.3 Definiciones 106
42.4 Reglas 107
42.4.1 Catalogo minimo de riesgos 107
42.4.2 Riesgos documentales especificos 107
42.4.3 Riesgos operativos especificos 107
42.5 Edge Cases 108
42.6 Invariantes 108
42.7 Preguntas abiertas 108
42.8 Decisiones pendientes 108

1 Mapa documental

1.1 Descripcion

Este documento explica como se organiza el repositorio de documentacion y que pregunta responde cada seccion. Su objetivo es reducir duplicidad, evitar mezclas de responsabilidad y facilitar el mantenimiento.

1.2 Alcance

Aplica a todo el repositorio de documentacion tecnica de Quiniela.

1.3 Definiciones

- **Seccion**: carpeta principal del repositorio, asociada a un area concreta del sistema.
- **Documento fuente**: archivo '.tex' editable dentro de una subcarpeta 'tex/'.
- **Entregable**: PDF publicado en la raiz de una seccion.
- **Documento maestro**: PDF que consolida los documentos aprobados del repositorio.

1.4 Reglas

Seccion	Pregunta principal que responde
`00_Overview`	Que es el sistema, como se organiza la documentacion y bajo que principios debe leerse.
`01_Producto_y_Reglas`	Cuales son las reglas oficiales del producto y del juego.
`02_Arquitectura`	Como se divide el sistema y cuales son sus limites principales.
`03_Backend`	Como se organiza el backend a nivel conceptual.
`04_Frontend`	Como se organiza el frontend a nivel conceptual.
`05_Datos`	Cual es el modelo de dominio y cuales son sus invariantes.
`06_Integraciones`	Como interactua el sistema con servicios externos.
`07_Seguridad_y_Operacion`	Que lineamientos operativos y de seguridad deben respetarse.
`08_Testing`	Como se valida la calidad funcional y tecnica.
`09_UI_UX`	Que principios de interfaz y experiencia guian el producto.
`10_Planeacion`	Que decisiones, riesgos y fases de trabajo siguen abiertas.

Una regla normativa debe definirse una sola vez en el repositorio. Los demas documentos deben referenciarla en lugar de duplicarla.

1.5 Edge Cases

- Si un tema encaja en dos secciones, se documenta en la seccion que lo define y se referencia desde la otra.
- Si una regla afecta varias capas del sistema, sigue perteneciendo primero al dominio, no a la capa tecnica que la ejecuta.

1.6 Invariantes

- Cada documento debe tener una sola responsabilidad primaria.
- Ninguna regla del negocio debe depender del lector interpretando el contexto.
- El documento maestro no reemplaza los documentos modulares; los consolida.

1.7 Preguntas abiertas

- Ninguna por ahora.

2 Vision y alcance

2.1 Descripcion

Quiniela es una plataforma web para administrar una quiniela del Mundial con registro publico, activacion manual por pago, picks por marcador exacto, resultados oficiales, calculo de puntajes y operacion administrativa con trazabilidad.

2.2 Alcance

Esta vision cubre el producto v1 y el limite funcional de la documentacion actual.

2.3 Definiciones

- **v1**: primera version operativa del producto para un solo torneo.
- **JD**: equipo administrativo responsable de operar la quiniela.
- **Quiniela oficial**: unica quiniela habilitada para el torneo cubierto en esta version.

2.4 Reglas

El sistema cubre los siguientes componentes funcionales:

- registro, login y recuperacion de contrasena;
- activacion por pago con comprobante y validacion manual;
- ingreso de picks por partido mediante marcador exacto;
- lock automatico previo al kickoff;
- importacion de resultados desde API externa con fallback manual;
- calculo de puntos, desempates y leaderboard;
- panel administrativo para usuarios, partidos, resultados y recalculo;
- auditoria de acciones criticas;
- archivado del torneo en modo solo lectura.

Fuera de alcance para v1:

- app movil nativa;
- multiples quinielas privadas;
- multiples torneos activos en paralelo;
- pasarela de pago;
- notificaciones por WhatsApp o push;
- white-label multi-club.

2.5 Edge Cases

- El producto debe quedar preparado para crecer en datos, pero la v1 no habilita multi-torneo concurrente.
- El volumen esperado es acotado, pero la documentacion no debe depender de ese volumen para definir reglas.

2.6 Invariantes

- La plataforma se documenta como sistema web con dos repos de implementacion y uno exclusivo de documentacion.
- El dominio del juego tiene prioridad sobre las decisiones de tecnologia.
- Todo comportamiento fuera de alcance debe quedar explicitamente fuera de v1.

2.7 Preguntas abiertas

- Ninguna por ahora.

3 Principios de ingenieria y arquitectura

3.1 Descripcion

Este documento fija los principios que deben reflejarse en la documentacion y, posteriormente, en la implementacion del sistema. No es una lista decorativa; actua como criterio de calidad para escribir reglas, contratos y limites.

3.2 Alcance

Aplica a backend, frontend, datos, integraciones y operacion.

3.3 Definiciones

- **KISS**: la solucion documental y tecnica debe ser lo mas simple posible sin sacrificar claridad.
- **Single Responsibility**: cada documento y cada modulo deben tener una responsabilidad dominante.
- **Explicit Contracts**: toda frontera debe declarar que recibe, que entrega y bajo que reglas opera.

3.4 Reglas

Principios de ingenieria aplicados al repositorio:

- 'KISS': los documentos deben ser directos y libres de teoria innecesaria.
- 'SOLID' y 'Single Responsibility': un documento responde una sola pregunta principal.
- 'DRY': una regla normativa se define una vez y luego se referencia.
- 'YAGNI': no se documentan capacidades fuera del alcance confirmado.
- 'Separation of Concerns': producto, dominio, arquitectura, datos, seguridad y operacion viven en documentos separados.
- 'Law of Demeter': se evita acoplar documentos a detalles de otras capas cuando no son necesarios.
- 'Fail Fast': se documentan estados invalidos, entradas invalidas y errores esperados.
- 'Least Privilege': los roles se definen con permisos minimos suficientes.
- 'Idempotency': recalculo, sincronizacion y acciones repetibles deben quedar documentadas como operaciones seguras frente a reintentos.
- 'Convention over Configuration': el repositorio adopta convenciones fijas para estructura, nombres y compilacion.
- 'Explicit Contracts': toda interfaz relevante debe quedar escrita de forma verificable.

Principios arquitectonicos confirmados:

- Frontend organizado por features.
- Separacion entre UI, logica y acceso a datos.
- 'shared' restringido a elementos realmente reutilizables.
- Backend como modular monolith.
- Separacion por dominio: auth, users, payments, matches, picks, results, scoring, leaderboard, admin, audit y notifications.
- Validacion en la frontera de entrada.
- Logica de negocio fuera de controllers.

Principios de representacion documental para arquitectura:

- La arquitectura no debe depender solo de texto corrido cuando existan limites o relaciones que puedan expresarse mejor visualmente.
- `02_Arquitectura` debe incluir diagramas de alto nivel y trees estructurales complementarios.
- `03_Backend` debe incluir al menos trees de modulos o carpetas con responsabilidad visible.
- `04_Frontend` debe incluir al menos trees por features y limites de `shared`.
- Un diagrama debe usarse cuando un tree no sea suficiente para explicar dependencias, flujos o contratos entre partes.

- Si un diagrama no agrega claridad real o genera alto costo de mantenimiento, se prefiere un tree claro y estable.

Lineamientos de produccion:

- 'Secure by Design'.
- Buenas practicas 'OWASP'.
- 'Twelve-Factor App'.
- Configuracion fuera del codigo.
- Logs como stream de eventos.
- Validacion estricta de inputs.
- Manejo claro de errores.
- Auditoria de acciones criticas.

Baseline aprobado:

- Frontend: Next.js, TypeScript, ESLint, Zod, TanStack Query, Tailwind CSS, Playwright.
- Backend: Node.js, Express, TypeScript, Zod, Prisma, Helmet, Pino, OpenTelemetry, Vitest.
- Base de datos: PostgreSQL.

3.5 Edge Cases

- Si una practica recomendada entra en conflicto con una regla del dominio, la regla del dominio no se altera sin decision explicita.
- Si una herramienta baseline cambia en el futuro, debe registrarse como decision del repositorio.

3.6 Invariantes

- La documentacion debe justificar limites y responsabilidades; no solo listarlos.
- Ningun contrato relevante debe quedar implicito.
- Las decisiones tecnicas deben preservar claridad operativa y trazabilidad.
- Arquitectura, backend y frontend deben mostrar su estructura de forma visual al menos mediante trees, y mediante diagramas cuando aporten claridad material.

3.7 Preguntas abiertas

- Ninguna por ahora.

4 Glosario

4.1 Descripcion

Este glosario define los terminos base del proyecto para evitar ambiguedades entre documentos.

4.2 Alcance

Aplica a todo el repositorio.

4.3 Definiciones

Termino	Definicion
Usuario pendiente	Persona registrada que aun no puede participar.
Usuario activo	Persona habilitada para ingresar picks y competir.
Usuario rechazado	Persona cuya activacion fue denegada durante el flujo de alta.
Usuario bloqueado	Persona cuya participacion fue revocada por accion administrativa.
Kickoff	Fecha y hora oficial de inicio del partido en el sistema.
Pick	Prediccion del usuario para un partido.
Marcador exacto	Prediccion de goles local y goles visita.
Outcome	Resultado del partido: gana local, empate o gana visita.
Lock	Momento a partir del cual un pick deja de ser editable.
Resultado oficial	Resultado utilizado para calcular puntajes.
Override manual	Correccion administrativa explicitamente auditada.
Leaderboard	Tabla de posiciones general o filtrada.
Fase KO	Cualquier fase eliminatoria posterior a grupos.
Archivado	Estado de cierre del torneo en modo solo lectura.

4.4 Reglas

- Un termino definido aqui debe usarse de forma consistente en el resto del repositorio.
- Si un termino cambia de significado, el glosario se actualiza primero.

4.5 Edge Cases

- Si dos areas usan palabras distintas para el mismo concepto, prevalece el termino normalizado en este glosario.

4.6 Invariantes

- No se usan sinonimos funcionales cuando puedan introducir ambiguedad.

4.7 Preguntas abiertas

- Ninguna por ahora.

5 Fuentes y gobernanza

5.1 Descripcion

Este documento define de donde salen las reglas del proyecto, como se resuelven contradicciones y como se registra una decision nueva.

5.2 Alcance

Aplica a toda incorporacion, actualizacion o correccion documental.

5.3 Definiciones

- **Fuente primaria**: insumo con autoridad para definir una regla.
- **Registro documental**: contenido vigente de este repositorio, incluyendo changelog y documentos actualizados.
- **Decision reciente**: instruccion explicita posterior a los documentos fuente.

5.4 Reglas

Jerarquia oficial:

1. decision mas reciente del responsable del proyecto;
2. registro documental vigente de este repositorio;
3. PRD;
4. backlog/UI-UX como apoyo de detalle.

Clasificacion obligatoria de vacios:

- ‘Decision pendiente‘;
- ‘Bloqueante‘;
- ‘Riesgo‘.

Criticidad obligatoria:

- ‘Bloquea backend‘;
- ‘Bloquea frontend‘;
- ‘Bloquea ambos‘;
- ‘No bloqueante‘.

Proceso documental:

1. detectar definiciones existentes;
2. detectar contradicciones o huecos;
3. registrar pendiente con criticidad;
4. cerrar la decision;
5. actualizar documento fuente;
6. registrar el cambio en ‘CHANGELOG.md‘.

5.5 Edge Cases

- Si una decision nueva contradice el PRD, no se corrige el PDF original; se actualiza este repositorio y el changelog.
- Si una definicion sigue abierta pero no bloquea el avance completo, se documenta el hueco y se continua alrededor de el.

5.6 Invariantes

- Ninguna contradiccion relevante debe quedar silenciosa.
- Toda decision nueva debe poder rastrearse en el repositorio.

5.7 Preguntas abiertas

- Ninguna por ahora.

6 Reglamento v1

6.1 Descripcion

Este documento establece el reglamento oficial de participacion de la quiniela v1 del Mundial. Su funcion es fijar las reglas base del juego de forma clara, estable y verificable, sin entrar en detalle tecnico de implementacion.

6.2 Alcance

Este reglamento aplica a:

- la unica quiniela oficial habilitada para el torneo en v1;
- todos los participantes registrados y activados;
- la operacion administrativa del torneo en aquello que afecta directamente la validez de la competencia.

Este documento no reemplaza los documentos de detalle sobre estados, picks, scoring o Edge Cases. Cuando una regla requiera precision operativa adicional, se complementa con los documentos especializados de la misma seccion.

6.3 Definiciones

- **Participante**: usuario en estado `ACTIVO` habilitado para competir.
- **Pick**: prediccion de un participante para un partido.
- **Marcador exacto**: prediccion de goles del equipo local y del equipo visita.
- **Outcome**: resultado del partido en terminos de victoria local, empate o victoria visita.
- **Lock**: momento a partir del cual un pick deja de ser editable.

- **Resultado oficial**: marcador valido para puntaje conforme a la regla de 90 minutos mas tiempo agregado.
- **Predicciones especiales**: predicciones unicas de Campeon y Goleador para el torneo.

6.4 Reglas

6.4.1 Participacion

1. La participacion inicia con registro publico en la plataforma.
2. La activacion del participante requiere validacion manual de pago con comprobante.
3. Solo un usuario en estado `ACTIVO` puede competir.
4. Un usuario que no este en estado `ACTIVO` no puede generar picks validos ni acumular puntaje competitivo.

6.4.2 Picks por partido

1. Cada pick por partido consiste en un marcador exacto.
2. El participante puede crear o modificar su pick mientras el partido permanezca abierto.
3. El lock ocurre automaticamente 5 minutos antes del kickoff oficial del partido.
4. A partir del lock, el pick queda cerrado y no puede modificarse.
5. Si el participante no dejo pick antes del lock, ese partido se considera sin pick y otorga 0 puntos.

6.4.3 Resultado oficial del partido

1. El resultado oficial para puntaje corresponde a los 90 minutos reglamentarios mas tiempo agregado.
2. El tiempo extra y los penales no alteran el puntaje de la quiniela.
3. En fases eliminatorias, un empate al final de 90 minutos mas tiempo agregado cuenta como empate para efectos de la quiniela.

6.4.4 Scoring

1. Acertar el outcome otorga 3 puntos.
2. Acertar el marcador exacto otorga 2 puntos adicionales.
3. El puntaje base maximo por partido es 5 puntos.
4. Los partidos aplican multiplicador segun la fase oficial del torneo.
5. Las predicciones especiales forman parte de v1.
6. Las predicciones especiales de Campeon y Goleador pueden crearse o modificarse hasta 5 minutos antes del kickoff oficial del primer partido del torneo.
7. A partir de ese momento, las predicciones especiales quedan bloqueadas.
8. Campeon correcto otorga 20 puntos.

9. Goleador correcto otorga 15 puntos.

Multiplicadores oficiales por fase:

Fase	Multiplicador
Grupos	x1.0
Octavos de final	x1.25
Cuartos de final	x1.5
Semifinales	x1.75
Tercer lugar	x1.75
Final	x2.0

6.4.5 Leaderboard y visibilidad

1. El torneo mantiene leaderboard general y vistas filtradas por jornada o fase.
2. Los picks de otros participantes permanecen ocultos mientras el partido no haya cerrado.
3. Los picks de otros participantes solo pueden hacerse visibles cuando el partido ya haya cerrado.

6.4.6 Cierre del torneo

1. Al finalizar el torneo, la quiniela pasa a estado archivado.
2. Un torneo archivado queda en modo solo lectura.

6.5 Edge Cases

- Si el kickoff oficial cambia antes del lock, el lock debe recalcularse con la nueva hora.
- Si un partido entra en estado suspendido, postergado, cancelado o corregido, aplican las reglas especificas del documento `01_05_Casos_Borde`.
- Si un resultado oficial se corrige despues de haber sido publicado, el puntaje y el leaderboard deben recalcularse conforme a la verdad oficial vigente.

6.6 Invariantes

- Ningun usuario no activo participa validamente en la competencia.
- Ningun pick bloqueado puede modificarse.
- Todo puntaje por partido usa exactamente el mismo criterio de resultado oficial.
- El puntaje base por partido nunca excede 5 puntos antes de multiplicadores.
- La visibilidad de picks ajenos nunca ocurre antes del cierre del partido correspondiente.

6.7 Preguntas abiertas

- No hay preguntas abiertas activas en este documento de reglamento; las definiciones operativas abiertas viven en los documentos especializados de la misma seccion.

6.8 Decisiones pendientes

- ‘Decision pendiente‘ ‘No bloqueante‘: confirmar si la vista reglamentaria publica necesita incluir una version resumida para participantes y una version extendida para operacion interna.

7 Estados y flujos

7.1 Descripcion

Este documento define los estados funcionales del sistema y las transiciones validas entre ellos. Su foco es el dominio, no la implementacion tecnica de pantalla o endpoint.

7.2 Alcance

Aplica a usuarios, partidos, picks, resultados y torneo.

7.3 Definiciones

- **Transicion**: cambio valido entre dos estados.
- **Estado terminal**: estado desde el cual no se espera retorno ordinario.
- **Proceso de revision de pago**: evaluacion administrativa del comprobante previo a la activacion del usuario.

7.4 Reglas

Estados de usuario:

- ‘PENDIENTE‘: registrado, con activacion aun no aprobada.
- ‘ACTIVO‘: habilitado para participar.
- ‘RECHAZADO‘: activacion denegada.
- ‘BLOQUEADO‘: acceso revocado por accion administrativa.

Reglas de interpretacion del estado de usuario:

- `EN_REVISION` no se define como estado de usuario en este documento.
- El proceso de revision del comprobante de pago pertenece al flujo administrativo de activacion y no altera por si mismo el estado funcional del usuario fuera de `PENDIENTE`.
- Mientras la activacion no haya sido aprobada, el usuario permanece en `PENDIENTE`.

Transiciones validas de usuario:

- registro nuevo a ‘PENDIENTE‘;
- ‘PENDIENTE‘ a ‘ACTIVO‘;
- ‘PENDIENTE‘ a ‘RECHAZADO‘;
- ‘ACTIVO‘ a ‘BLOQUEADO‘.

Estados de partido a nivel funcional:

- ‘PROGRAMADO‘;
- ‘BLOQUEADO‘ para picks;
- ‘CERRADO‘;
- ‘SUSPENDIDO‘;
- ‘POSTERGADO‘;
- ‘CANCELADO‘;
- ‘CORREGIDO‘ como marca de resultado ajustado.

Transiciones validas de partido:

- ‘PROGRAMADO‘ a ‘BLOQUEADO‘;
- ‘BLOQUEADO‘ a ‘CERRADO‘;
- ‘PROGRAMADO‘ o ‘BLOQUEADO‘ a ‘POSTERGADO‘;
- ‘BLOQUEADO‘ o ‘CERRADO‘ a ‘SUSPENDIDO‘;
- ‘PROGRAMADO‘, ‘BLOQUEADO‘ o ‘SUSPENDIDO‘ a ‘CANCELADO‘;
- ‘CERRADO‘ a ‘CORREGIDO‘.

Estados de pick:

- ‘ABIERTO‘: editable;
- ‘BLOQUEADO‘: no editable;
- `SIN_PICK`: no existe prediccion guardada al momento del lock.

Estados de torneo:

- ‘ACTIVO‘;
- ‘ARCHIVADO‘.

7.5 Edge Cases

- Un usuario rechazado no se reactiva sin una accion administrativa explicita y documentada.
- Un usuario en `PENDIENTE` puede tener actividad administrativa sobre su comprobante sin cambiar todavia de estado funcional.
- Un partido corregido conserva su historial previo mediante auditoria; la correccion no borra la trazabilidad.
- `SIN_PICK` no es una prediccion; es la ausencia valida de una prediccion antes del cierre.

7.6 Invariantes

- Un usuario no puede ser simultaneamente ‘ACTIVO‘ y ‘BLOQUEADO‘.
- Todo usuario no activado permanece en `PENDIENTE` hasta una resolucion administrativa explicita.

- Un torneo archivado no vuelve a estado activo dentro del flujo ordinario.
- Un pick bloqueado no vuelve a estado abierto por flujo ordinario.

7.7 Preguntas abiertas

- No hay preguntas abiertas activas en este documento despues de cerrar la separacion entre estado de usuario y revision administrativa del pago.

7.8 Decisiones pendientes

- ‘Decision pendiente‘ ‘No bloqueante‘: si el backend modela el comprobante de pago como entidad propia, sus estados internos deben documentarse fuera de este documento para no mezclar ejes de estado.

8 Picks y locks

8.1 Descripcion

Este documento define las reglas funcionales de captura, validacion, bloqueo y visibilidad de los picks del sistema. Su objetivo es dejar sin ambiguedad que puede ingresarse, hasta cuando puede modificarse y bajo que condiciones deja de ser visible o editable.

8.2 Alcance

Aplica a:

- picks por partido;
- predicciones especiales de Campeon y Goleador;
- reglas de lock asociadas a ambos tipos de prediccion.

8.3 Definiciones

- **Ventana abierta**: periodo durante el cual una prediccion puede crearse o editarse.
- **Lock**: momento a partir del cual una prediccion deja de ser editable.
- **Picks por partido**: predicciones repetibles asociadas a partidos individuales.
- **Predicciones especiales**: predicciones unicas de torneo para Campeon y Goleador.
- **Sin pick**: ausencia valida de prediccion guardada antes del lock.

8.4 Reglas

8.4.1 Reglas generales

1. Solo usuarios en estado `ACTIVO` pueden guardar picks validos.
2. Toda validacion de lock debe depender del kickoff oficial vigente en el sistema y no del reloj local del navegador.

3. El backend debe rechazar cualquier modificacion posterior al lock, aunque la interfaz intente enviarla.

8.4.2 Picks por partido

1. Cada pick por partido se compone de dos valores enteros: goles local y goles visita.
2. El rango valido para ambos valores es de 0 a 9 inclusive.
3. Mientras el partido este en ventana abierta, el participante puede crear o editar su pick.
4. El lock del pick por partido ocurre automaticamente 5 minutos antes del kickoff oficial vigente del partido.
5. Si no existe pick guardado al llegar el lock, el partido queda funcionalmente sin pick y otorga 0 puntos.

8.4.3 Predicciones especiales

1. Las predicciones especiales forman parte del modulo funcional de picks.
2. Las predicciones especiales se capturan en un flujo separado del flujo de picks por partido.
3. Las predicciones especiales pueden crearse o editarse hasta 5 minutos antes del kickoff oficial del primer partido del torneo.
4. A partir de ese momento, las predicciones especiales quedan bloqueadas.

8.4.4 Visibilidad

1. Los picks de otros usuarios permanecen ocultos mientras el partido no haya cerrado.
2. Los picks de otros usuarios solo se vuelven visibles despues de que el partido ya haya cerrado.
3. Este documento no fija aun la politica final de visualizacion de predicciones especiales frente a otros usuarios; esa decision se cerrara cuando se refine la visualizacion del leaderboard y perfiles.

8.5 Edge Cases

- Si el kickoff cambia antes del lock, el cierre debe recalcularse con la nueva hora oficial.
- Si el kickoff cambia despues del lock por una postergacion, el pick ya bloqueado permanece valido salvo decision administrativa extraordinaria auditada.
- Si el usuario guarda parcialmente una jornada, solo cuentan los partidos cuya prediccion haya sido persistida antes del lock correspondiente.
- Si el sistema no encuentra un pick guardado para un partido al llegar el lock, no debe generar una prediccion implicita ni un valor por defecto.

8.6 Invariantes

- No existe edicion valida de picks por partido despues del lock.

- No existe edicion valida de predicciones especiales despues de su fecha de cierre.
- La visibilidad de picks ajenos nunca ocurre antes del cierre del partido correspondiente.
- Un partido sin pick no genera prediccion automatica.
- Ningun valor de goles fuera del rango 0 a 9 es valido.

8.7 Preguntas abiertas

- Confirmar la politica exacta de visibilidad de Campeon y Goleador frente a otros usuarios.

8.8 Decisiones pendientes

- 'Decision pendiente' 'Bloquea frontend': definir como se presenta en UI el flujo separado de predicciones especiales dentro del modulo de picks.
- 'Decision pendiente' 'No bloqueante': confirmar si el estado funcional 'sin pick' se documentara tambien como estado persistido explicito o solo como resultado de negocio.

9 Scoring y desempates

9.1 Descripcion

Este documento define el calculo oficial de puntajes del torneo, el uso de multiplicadores, la integracion de predicciones especiales y el orden de desempates del leaderboard.

9.2 Alcance

Aplica a:

- puntaje por partido;
- puntaje acumulado del torneo;
- predicciones especiales de Campeon y Goleador;
- ordenamiento del leaderboard.

9.3 Definiciones

- **Puntaje base**: suma de puntos obtenidos antes de aplicar multiplicador.
- **Puntaje acumulado**: suma total de puntajes validos del participante en el torneo.
- **Desempate**: regla de ordenamiento entre participantes con el mismo puntaje acumulado.
- **Desglose**: separacion visible del origen de puntos sin alterar el total oficial.

9.4 Reglas

9.4.1 Puntaje por partido

- Acertar el outcome otorga 3 puntos.

- Acertar el marcador exacto otorga 2 puntos adicionales.
- El puntaje base maximo por partido es 5 puntos.

9.4.2 Multiplicadores por fase

Fase	Multiplicador
Grupos	x1.0
Octavos de final	x1.25
Cuartos de final	x1.5
Semifinales	x1.75
Tercer lugar	x1.75
Final	x2.0

- El multiplicador se aplica al puntaje del partido conforme a la fase oficial del torneo.
- El calculo conserva precision exacta al aplicar el multiplicador.
- La logica interna de scoring no debe redondear el valor calculado.
- La representacion visual del puntaje puede formatearse con precision controlada, pero ese formato no modifica el valor oficial de calculo.

9.4.3 Predicciones especiales

- Campeon correcto otorga 20 puntos.
- Goleador correcto otorga 15 puntos.
- Las predicciones especiales forman parte del puntaje total acumulado del torneo.
- Las predicciones especiales no crean un leaderboard separado por defecto.

9.4.4 Leaderboard

- El leaderboard se ordena por puntaje total acumulado.
- El puntaje total acumulado integra:
  - puntaje de picks por partido;
  - puntaje de predicciones especiales.
- El sistema debe poder mostrar desglose del origen de puntos sin alterar el total oficial.
- Las vistas esperadas del leaderboard son:
  - general;
  - por jornada;
  - por fase.

9.4.5 Desempates

Orden oficial de desempates:

1. mayor cantidad de marcadores exactos acertados;
2. mayor cantidad de outcomes acertados;
3. mayor puntaje acumulado en fase KO;
4. fecha y hora de registro mas temprana.

9.5 Edge Cases

- Si un resultado oficial se corrige, el puntaje afectado debe recalcularse de forma deterministica.
- Si un partido fue cancelado sin resultado oficial utilizable, ese partido no altera el ranking.
- Si el formato visual del puntaje usa redondeo o truncamiento para mostrar datos, ese formato no puede alterar el orden real del leaderboard.

9.6 Invariantes

- Dos participantes con el mismo puntaje total deben ordenarse siempre con el mismo algoritmo de desempate.
- El recalculo no puede producir duplicacion de puntos.
- El leaderboard refleja solo resultados oficiales vigentes.
- El total acumulado es la unica base oficial de ranking.
- Las predicciones especiales suman al total, aunque puedan mostrarse en desglose.

9.7 Preguntas abiertas

- Confirmar la precision visual exacta con la que se mostraran puntajes no enteros en la interfaz.

9.8 Decisiones pendientes

- ‘Decision pendiente‘ ‘Bloquea frontend‘: definir como se presenta visualmente el desglose de puntos en leaderboard, perfil y detalle de rendimiento.
- ‘Decision pendiente‘ ‘No bloqueante‘: confirmar si el leaderboard por jornada excluye visualmente las predicciones especiales aunque el total general las incluya.

10 Edge Cases

10.1 Descripcion

Este documento define los escenarios funcionales no triviales que pueden afectar justicia competitiva, trazabilidad, recalc ulo o intervencion administrativa. Su objetivo es evitar interpretaciones ad hoc cuando el comportamiento ordinario del torneo no sea suficiente.

10.2 Alcance

Aplica a:

- partidos suspendidos, postergados, cancelados o corregidos;
- correcciones de resultados;
- prioridad entre fuentes de resultado;
- comunicacion funcional al usuario cuando el resultado vigente cambie.

10.3 Definiciones

- **Suspendido**: partido interrumpido temporalmente sin resultado final utilizable.
- **Postergado**: partido movido a una nueva fecha.
- **Cancelado**: partido sin resultado oficial utilizable para la quiniela.
- **Corregido**: resultado oficial ajustado posteriormente.
- **Override manual**: correccion administrativa autorizada sobre el resultado de un partido.

10.4 Reglas

10.4.1 Partidos postergados

1. Si un partido se posterga antes del lock, el lock se recalcula con el nuevo kickoff oficial.
2. Si un partido se posterga despues del lock, los picks ya guardados permanecen validos y bloqueados, salvo decision administrativa extraordinaria auditada.

10.4.2 Partidos suspendidos

1. Si un partido se suspende y luego se completa con resultado oficial compatible con la regla de 90 minutos mas tiempo agregado, el sistema puntua cuando ese resultado exista.
2. Mientras no exista un resultado oficial utilizable, el partido no debe cerrar scoring definitivo.

10.4.3 Partidos cancelados

1. Si un partido se cancela y no existe resultado oficial utilizable para la quiniela, no genera puntaje para ningun usuario.
2. Un partido cancelado no debe alterar ranking, desempates ni estadisticas de acierto del torneo.

10.4.4 Resultados corregidos

1. Si un resultado se corrige por fuente valida o por override manual autorizado, el sistema debe recalcular el puntaje afectado y dejar trazabilidad completa.
2. Toda correccion manual de resultado debe registrar actor, momento, valor anterior, valor nuevo y justificacion.
3. Cuando exista un override manual autorizado, ese valor se considera la verdad oficial vigente hasta que una nueva accion explicita y auditada lo modifique.
4. La sincronizacion automatica de la API no debe sobrescribir un override manual vigente sin una accion administrativa explicita.

10.4.5 Comunicacion funcional al usuario

1. Cuando un resultado oficial previamente publicado sea corregido, el sistema debe comunicar al usuario que el resultado vigente cambio y que los puntajes fueron recalculados.
2. El mensaje funcional al usuario debe ser explicito, breve y orientado a transparencia.
3. El mensaje no necesita mostrar por defecto el valor anterior y el valor nuevo; ese nivel de detalle pertenece a auditoria y operacion administrativa.

10.5 Edge Cases

- Si el proveedor de resultados entrega primero un valor temporal y luego otro definitivo, solo el resultado oficial vigente debe permanecer activo.
- Si el sistema recalcula varias veces el mismo partido corregido, el proceso debe seguir siendo idempotente.
- Si un usuario disputa un cambio de puntaje, la evidencia operativa debe salir de la auditoria y de la fuente oficial vigente aplicada.
- Si una correccion manual reemplaza un dato de API y luego la API vuelve a sincronizar, el valor manual no debe perderse por sincronizacion automatica.

10.6 Invariantes

- Ninguna correccion manual debe borrar la historia del valor anterior.
- Ningun caso borde debe resolverse fuera de trazabilidad.
- Un partido cancelado no asigna puntos.
- Un override manual vigente tiene prioridad operativa sobre la API automatica hasta nueva accion explicita.

10.7 Preguntas abiertas

- Confirmar en que superficies de la interfaz debe mostrarse el mensaje de resultado corregido: detalle de partido, mis picks, leaderboard o varias de ellas.

10.8 Decisiones pendientes

- ‘Decision pendiente‘ ‘Bloquea frontend‘: definir exactamente donde y durante cuanto tiempo se muestra el aviso de resultado corregido al usuario.
- ‘Decision pendiente‘ ‘No bloqueante‘: confirmar si la UI debe distinguir visualmente entre resultado corregido por API y resultado corregido por override manual.

11 Vision arquitectonica

11.1 Descripcion

La solucion se documenta como un sistema web dividido en tres repositorios: frontend, backend y documentacion. El dominio del juego es el centro del sistema y condiciona el diseno de las demas capas, contratos y procesos de operacion.

11.2 Alcance

Esta vista describe componentes, limites y principios de separacion. No reemplaza la documentacion detallada de backend ni frontend.

11.3 Definiciones

- **Repositorio frontend**: aplicacion web construida en Next.js.
- **Repositorio backend**: API y logica de negocio construida en Node.js y Express.
- **Repositorio documental**: fuente de verdad de reglas, dominio y criterios del sistema.
- **Modular monolith**: backend dividido por modulos de dominio con despliegue unico.

11.4 Reglas

Vista de alto nivel:

- Frontend consume al backend exclusivamente mediante contratos de API.
- Backend concentra autenticacion, autorizacion, reglas de negocio, integraciones y persistencia.
- PostgreSQL almacena usuarios, comprobantes, partidos, picks, resultados, puntajes y auditoria.
- Una API externa provee resultados, con capacidad de override manual.
- La documentacion define reglas antes de que se cierren interfaces de implementacion.

11.4.1 Vista logica de ejecucion

Usuarios y administradores	→	Frontend web	→	Backend API y dominio
			↘	PostgreSQL como sistema de registro
			↗	API externa de resultados y override manual auditado

Interpretacion de la vista:

- El frontend es la unica superficie interactiva para participantes y administradores.
- El backend concentra las reglas oficiales del juego, incluso cuando la interfaz muestre validaciones preliminares.
- La base de datos conserva el estado persistente del sistema y la evidencia necesaria para recalculo y auditoria.

- La integracion externa de resultados no define por si sola la verdad oficial vigente cuando exista override manual autorizado.

11.4.2 Tree conceptual de repositorios

El siguiente tree es conceptual y describe distribucion de responsabilidades, no nombres fisicos definitivos de carpetas:

```
quiniela/
|-- frontend/
|   |-- aplicacion web para participantes y administradores
|   |-- UI, navegacion y consumo de API
|   `-- validacion de entrada y manejo de estado cliente
|-- backend/
|   |-- API HTTP
|   |-- modulos de dominio
|   |-- integraciones externas
|   `-- persistencia y auditoria
`-- documentacion/
    |-- reglas de negocio
    |-- arquitectura y datos
    `-- decisiones y trazabilidad documental
```

Principios estructurales:

- El dominio no debe depender de detalles de UI.
- La logica de negocio no debe vivir en controllers.
- La validacion de entradas ocurre en la frontera de entrada.
- Las acciones criticas deben quedar auditadas.
- El sistema debe poder recalcular sin romper consistencia.

Limites de responsabilidad:

- El frontend es responsable de experiencia de usuario, composicion de vistas y validaciones de entrada que mejoren usabilidad.
- El backend es responsable de enforcement real de permisos, locks, scoring, desempates, overrides y auditoria.
- La documentacion es responsable de definir reglas, vocabulario y limites antes de que las implementaciones los materialicen.
- Las integraciones externas quedan encapsuladas en backend y no deben filtrarse como dependencia directa al frontend.

11.5 Edge Cases

- Si una necesidad de implementacion entra en conflicto con una regla del dominio, la regla se revisa documentalmente antes de ajustarse en codigo.
- Si el proveedor externo de resultados cambia, la arquitectura debe preservar el contrato interno

del dominio.

11.6 Invariantes

- El frontend no accede directamente a la base de datos.
- El backend es responsable de enforcement de locks y scoring.
- La documentacion es el origen de definiciones funcionales.
- Ninguna integracion externa puede modificar directamente el estado oficial del sistema sin pasar por las reglas del backend.

11.7 Preguntas abiertas

- Ninguna por ahora.

12 Arquitectura frontend

12.1 Descripcion

Este documento describe la arquitectura objetivo del frontend para v1. Su enfoque es una organizacion por features con separacion entre UI, logica y acceso a datos, y un uso restringido de `shared` a piezas autenticamente reutilizables.

12.2 Alcance

Describe la arquitectura del repositorio frontend, no su implementacion detalle a detalle.

12.3 Definiciones

- **Feature**: unidad funcional autocontenida del frontend.
- **Shared**: espacio comun reservado para elementos verdaderamente reutilizables.
- **Estado remoto**: datos obtenidos del backend y sincronizados en cliente.
- **Estado local**: estado efimero de interfaz que no representa verdad persistente del dominio.

12.4 Reglas

- El frontend seguira un esquema simplificado de organizacion por features.
- La logica de negocio del juego no se resolvera exclusivamente en cliente.
- Los contratos de entrada y salida se validaran con Zod donde aplique.
- El estado remoto se gestionara como una preocupacion distinta del estado local de UI.
- `shared` no debe convertirse en un contenedor ambiguo de logica sin ownership claro.

12.4.1 Capas funcionales esperadas

- Capa de presentacion: componentes de pantalla, layout y estados visuales.
- Capa de feature: logica de interaccion propia de cada flujo funcional.
- Capa de acceso a datos: queries, mutaciones y adaptacion de contratos del backend.
- Capa compartida: primitivas reutilizables, utilidades neutrales y convenciones comunes.

12.4.2 Tree arquitectonico sugerido

El siguiente tree es una referencia descriptiva de organizacion, no una imposicion literal de nombres finales:

```
frontend/
|-- app/
|   |-- rutas, layouts y composicion de pantallas
|   `-- boundaries de navegacion
|-- features/
|   |-- auth/
|   |-- activation/
|   |-- picks/
|   |-- special-predictions/
|   |-- leaderboard/
|   |-- profile/
|   `-- admin/
|-- entities/
|   |-- user/
|   |-- match/
|   |-- pick/
|   |-- result/
|   `-- leaderboard-entry/
`-- shared/
    |-- ui/
    |-- lib/
    |-- config/
    `-- contracts/
```

12.4.3 Responsabilidades por bloque

- `app`: define rutas, layouts y composicion de alto nivel.
- `features`: encapsula comportamiento orientado a objetivos del usuario o del admin.
- `entities`: concentra modelos de presentacion y adaptaciones cercanas al dominio reutilizables por multiples features.
- `shared`: contiene piezas neutrales sin dependencia de un flujo especifico.

12.4.4 Criterios de uso de shared

- Una pieza solo entra en `shared` si es reusable en mas de un feature sin conocimiento de negocio especifico.
- Una validacion propia de picks, activacion o scoring no pertenece a `shared`; pertenece al feature o entidad correspondiente.
- Un componente visual generico puede vivir en `shared/ui` si no codifica reglas de dominio.

12.5 Edge Cases

- Si una validacion existe en frontend por experiencia de usuario, backend sigue siendo la autoridad final de enforcement.
- Si una feature necesita compartir demasiada logica con otra, debe revisarse si esa logica es de entidad comun o si la frontera entre features esta mal definida.

12.6 Invariantes

- La arquitectura frontend no redefine reglas oficiales del dominio ya cerradas en la seccion `01_Producto_y_Reglas`.
- Ninguna pantalla debe asumir permisos, locks o scoring sin respaldo del backend.
- El frontend no accede directamente a base de datos ni a la API externa de resultados.

12.7 Preguntas abiertas

- Confirmar si `special-predictions` vive como feature independiente o como subflujo interno de `picks` en la estructura final del frontend.

12.8 Decisiones pendientes

- `Decision pendiente` `Bloquea frontend`: confirmar el inventario final de features visibles en v1 para aterrizar la estructura real del repositorio frontend.
- `Decision pendiente` `No bloqueante`: decidir si el frontend requerira una carpeta `entities` separada o si la complejidad de v1 permite resolver esa capa dentro de features y `shared`.

13 Arquitectura backend

13.1 Descripcion

Este documento describe la arquitectura objetivo del backend para v1. Su enfoque es un modular monolith por dominio, con validacion en fronteras de entrada, logica fuera de controllers y separacion clara entre dominio, integraciones y preocupaciones transversales.

13.2 Alcance

Describe la arquitectura del repositorio backend a nivel conceptual.

13.3 Definiciones

- **Modulo**: agrupacion de responsabilidad de dominio dentro del backend.
- **Frontera de entrada**: punto donde el sistema recibe solicitudes y valida contratos.
- **Servicio transversal**: capacidad compartida que asiste a multiples modulos sin redefinir sus reglas de dominio.

13.4 Reglas

- El backend seguira una arquitectura modular por dominio.
- La logica de negocio critica vivira fuera de controllers.
- Las integraciones externas no deben contaminar el dominio con contratos ajenos.
- Cada modulo debe exponer una responsabilidad clara y dependencias limitadas.
- La persistencia debe servir al dominio y no imponer reglas funcionales por omision.

13.4.1 Modulos de dominio esperados

- `auth`: autenticacion y sesion.
- `users`: identidad y estado funcional de cuenta.
- `payments`: comprobantes, revision administrativa y activacion.
- `matches`: calendario, kickoff, fases y estado operativo del partido.
- `picks`: captura, edicion y bloqueo de picks.
- `results`: resultados oficiales, sincronizacion externa y overrides.
- `scoring`: calculo de puntos, multiplicadores y desempates.
- `leaderboard`: vistas de ranking y consolidaciones derivadas.
- `admin`: acciones administrativas sobre el sistema.
- `audit`: eventos auditables y trazabilidad.
- `notifications`: correos y eventos comunicables.

13.4.2 Capas internas esperadas

- Capa de entrada: controllers, handlers o routers con validacion de frontera.
- Capa de aplicacion: coordinacion de casos de uso y secuencias de proceso.
- Capa de dominio: reglas de negocio, invariantes y decisiones funcionales.
- Capa de infraestructura: persistencia, integraciones externas, correo, logging y observabilidad.

13.4.3 Tree arquitectonico sugerido

El siguiente tree es una referencia descriptiva del modular monolith y no obliga nombres fisicos exactos:

```
backend/
|-- src/
|   |-- modules/
|   |   |-- auth/
|   |   |-- users/
|   |   |-- payments/
|   |   |-- matches/
|   |   |-- picks/
|   |   |-- results/
|   |   |-- scoring/
|   |   |-- leaderboard/
|   |   |-- admin/
|   |   |-- audit/
|   |   `-- notifications/
|   |-- shared/
|   |   |-- config/
|   |   |-- db/
|   |   |-- logging/
|   |   |-- observability/
|   |   `-- contracts/
|   `-- app/
|       |-- server bootstrap
|       `-- cross-cutting wiring
```

13.4.4 Servicios transversales esperados

- configuracion;
- acceso a base de datos;
- logging estructurado;
- observabilidad;
- manejo comun de errores;
- utilidades de validacion y contratos compartidos.

13.4.5 Limites de responsabilidad

- Controllers o routers reciben solicitudes, validan y delegan; no deciden reglas del juego.
- El dominio decide activacion, locks, scoring, desempates y prioridad entre override manual y API.
- Infraestructura adapta Prisma, correo, logs y proveedor de resultados al contrato interno del backend.
- `admin` no redefine reglas del dominio; orquesta acciones administrativas autorizadas sobre modulos existentes.

13.5 Edge Cases

- Si una integracion externa cambia su contrato, la adaptacion debe quedar aislada en infraestructura o en el modulo de integracion correspondiente.
- Si una accion administrativa impacta multiples modulos, la coordinacion debe vivir en aplicacion sin romper ownership del dominio.

13.6 Invariantes

- Ningun modulo backend debe redefinir por su cuenta reglas oficiales de scoring, locks o activacion.
- Ninguna regla critica del juego debe depender de un controller o de una tarea externa sin pasar por el dominio.
- Ningun proveedor externo debe convertirse en fuente de verdad directa sin mediacion del backend.

13.7 Preguntas abiertas

- Confirmar si `notifications` vivira como modulo propio o como servicio transversal con ownership compartido.

13.8 Decisiones pendientes

- `Decision pendiente No bloqueante`: cerrar el limite exacto entre `admin` y los modulos de dominio al aterrizar procesos backend detallados.
- `Decision pendiente No bloqueante`: definir si el backend necesita una capa `shared/contracts` separada o si los contratos quedaran encapsulados por modulo.

14 Contratos y limites

14.1 Descripcion

Este documento delimita que puede saber, consumir y modificar cada capa del sistema. Su objetivo es evitar acoplamiento accidental entre dominio, frontend, backend, datos e integraciones, y dejar claras las fronteras que protegen la coherencia del producto.

14.2 Alcance

Aplica a los limites entre repositorios y entre capas internas principales.

14.3 Definiciones

- **Contrato**: acuerdo explicito de entrada, salida y comportamiento esperado.
- **Limite**: frontera de responsabilidad entre componentes.

14.4 Reglas

Contratos entre repositorios:

- El frontend consume datos del backend por HTTP y contratos aplicacionales.
- El frontend no depende directamente de la base de datos ni de la API externa de resultados.
- El backend usa la base de datos como sistema de registro y persistencia.
- El backend encapsula integraciones externas y expone al frontend una vista de dominio estable.

Matriz de limites entre capas principales:

Capa o repositorio	Puede depender de	No debe depender de
Frontend	Backend por contratos HTTP, definiciones funcionales del repositorio documental	Base de datos, API externa de resultados, reglas privadas de infraestructura
Backend dominio	Modelos y reglas internas del propio backend	Componentes de UI, contratos crudos del proveedor externo, decisiones visuales de frontend
Backend infraestructura	Base de datos, proveedores externos, correo, logging, observabilidad	Decisiones funcionales que pertenecen al dominio
Integraciones externas	Contratos del proveedor adaptados en backend	Frontend o reglas oficiales del juego
Documentacion	Fuentes del proyecto y decisiones aprobadas	Detalles de implementacion que contradigan reglas no cerradas

Limites internos esperados del backend:

- ‘auth‘: autenticacion y sesion.
- ‘users‘: identidad y estado de cuenta.
- ‘payments‘: recepcion y validacion funcional de comprobantes.
- ‘matches‘: calendario, kickoff y estados de partido.
- ‘picks‘: captura y lock de picks.
- ‘results‘: recepcion de resultados oficiales y overrides.
- ‘scoring‘: calculo de puntos y desempates.
- ‘leaderboard‘: vistas de ranking.
- ‘admin‘: acciones administrativas.
- ‘audit‘: trazabilidad.
- ‘notifications‘: correos y avisos del sistema.

Reglas de dependencia internas esperadas:

- `scoring` depende de resultados vigentes, picks validos y fases oficiales, pero no de detalles de interfaz.

- `leaderboard` consume scoring y datos de usuario, pero no redefine la formula del puntaje.
- `admin` coordina acciones autorizadas sobre modulos de dominio, pero no se convierte en una fuente paralela de reglas.
- `audit` y `notifications` pueden observar eventos de otros modulos, pero no deben introducir mutaciones funcionales no autorizadas.

Limites internos esperados del frontend:

- Las features consumen contratos del backend y modelos de presentacion, no queries directas a infraestructura.
- `shared` no absorbe logica propia de un feature como picks, activacion o scoring.
- La navegacion y la composicion de pantallas no redefinen permisos ni estados oficiales del dominio.

Contratos de documentacion:

- Las reglas funcionales se definen primero en `01_Producto_y_Reglas`.
- Los datos conceptuales se definen primero en `05_Datos`.
- Backend y frontend consumen estas definiciones, no las reemplazan.

Reglas de adaptacion en frontera:

- Todo contrato externo debe adaptarse al lenguaje del dominio antes de ser consumido por modulos internos.
- Todo contrato expuesto al frontend debe representar una vista estable del dominio y no detalles internos de infraestructura.
- Toda validacion de entrada en frontend mejora usabilidad, pero la validacion oficial vive en backend.

14.5 Edge Cases

- Si una integracion externa entrega un modelo incompatible con el dominio, se adapta en el limite de integracion, no en el frontend.
- Si una regla de negocio obliga a recalculo, el contrato del dominio prevalece sobre cualquier cache o vista derivada.
- Si un proceso administrativo necesita afectar multiples modulos, la coordinacion debe respetar ownership del dominio y dejar auditoria de cada cambio relevante.

14.6 Invariantes

- Ninguna capa consume directamente internals de otra sin contrato definido.
- Ninguna integracion externa define la logica oficial del juego por si sola.
- Ninguna vista derivada puede convertirse en sustituto de la fuente normativa del dominio.
- Ninguna validacion de frontend reemplaza la validacion oficial del backend.

14.7 Preguntas abiertas

- Confirmar si `notifications` vivira como modulo independiente o como servicio transversal con ownership distribuido.

14.8 Decisiones pendientes

- 'Decision pendiente' 'No bloqueante': confirmar el limite exacto entre 'admin' y los modulos de dominio cuando se cierre la arquitectura backend.

15 Modulos y responsabilidades

15.1 Descripcion

Este documento define los modulos esperados del backend y su reparto de responsabilidades. Su objetivo es dejar claro que le pertenece a cada modulo, que tipo de entradas y salidas maneja, y que dependencias son aceptables dentro del modular monolith.

15.2 Alcance

Aplica al repositorio backend y a su organizacion por dominio. No redefine reglas del negocio ya cerradas; solo distribuye ownership tecnico-funcional.

15.3 Definiciones

- **Modulo**: unidad de organizacion del backend alrededor de una responsabilidad de dominio.
- **Ownership**: responsabilidad principal de un modulo sobre una parte del comportamiento o de los datos del sistema.
- **Dependencia permitida**: relacion entre modulos aceptable dentro del diseno del backend.

15.4 Reglas

- El backend se documentara por modulos de dominio.
- Cada modulo debera declarar sus entradas, salidas y dependencias permitidas.
- La logica de negocio no se documentara como responsabilidad de controllers.
- Un modulo no debe absorber responsabilidades ajenas solo por conveniencia de implementacion.

15.4.1 Tree modular esperado

```
modules/
|-- auth/
|-- users/
|-- payments/
|-- matches/
|-- picks/
```

```
|-- results/
|-- scoring/
|-- leaderboard/
|-- admin/
|-- audit/
`-- notifications/
```

15.4.2 Responsabilidades por modulo

Modulo	Ownership principal	Entradas / salidas esperadas	Dependencias permitidas
`auth`	autenticacion, sesion y acceso inicial	credenciales, tokens, sesion valida o rechazada	`users`, servicios transversales
`users`	identidad y estado funcional de cuenta	registro, consulta y cambio de estado de usuario	`audit`, `notifications`
`payments`	comprobantes, revision y activacion administrativa	carga y revision de comprobante, resolucion administrativa	`users`, `audit`, `notifications`
`matches`	calendario, fases, kickoff y estado de partido	partidos, fases, horarios y cambios de estado	`audit`
`picks`	captura, edicion y lock de picks y predicciones especiales	picks validos o rechazados, estado de lock	`users`, `matches`, `audit`
`results`	resultado vigente, sincronizacion externa y overrides	resultados externos o manuales, verdad oficial vigente	`matches`, `audit`
`scoring`	calculo de puntos, multiplicadores y desempates	score entries, recalculo y total acumulado	`picks`, `results`, `matches`, `audit`
`leaderboard`	vistas derivadas del ranking	ranking general, por jornada o fase	`scoring`, `users`
`admin`	coordinacion de acciones administrativas autorizadas	comandos administrativos, ejecucion auditada	cualquier modulo de dominio por casos de uso explicitos
`audit`	trazabilidad de acciones y procesos	eventos auditables persistidos y consultables	servicio transversal observado por otros modulos
`notifications`	envio de correos y avisos funcionales	mensajes y disparadores de comunicacion	`users`, `payments`, `results`, `admin`

15.4.3 Reglas de colaboracion entre modulos

- `payments` no activa usuarios por mutacion directa informal; la activacion debe quedar trazada y reflejada en `users`.

- `picks` controla captura y lock, pero no calcula puntaje.
- `results` decide la verdad vigente del resultado, pero no recalcula ranking por si solo; esa responsabilidad pertenece a `scoring` y `leaderboard`.
- `leaderboard` consume resultados de scoring y no redefine desempates ni formulas de puntos.
- `admin` coordina, no reemplaza ownership de los modulos de dominio.

15.5 Edge Cases

- Si una accion administrativa impacta varios modulos, cada cambio relevante debe seguir perteneciendo a su modulo responsable y quedar auditado.
- Si `notifications` se implementa como servicio transversal en lugar de modulo independiente, su ownership funcional debe seguir documentado para no quedar difuso.

15.6 Invariantes

- Ningun modulo debe redefinir reglas centrales de `01_Producto_y_Reglas`.
- Ningun controller debe convertirse en responsable efectivo de reglas de dominio.
- Toda colaboracion entre modulos debe preservar un ownership principal identificable.

15.7 Preguntas abiertas

- Confirmar la frontera final entre `admin`, `audit` y `notifications` cuando se documenten procesos backend detallados.

15.8 Decisiones pendientes

- ‘Decision pendiente‘ ‘No bloqueante‘: decidir si `notifications` queda como modulo propio o como capacidad transversal con ownership distribuido.
- ‘Decision pendiente‘ ‘No bloqueante‘: confirmar si `audit` expone consultas administrativas como modulo propio o solo como infraestructura observable por otros modulos.

16 Procesos de negocio

16.1 Descripcion

Este documento describe los procesos backend clave que sostienen el comportamiento del sistema. Su objetivo es dejar claras las secuencias funcionales, sus disparadores, validaciones y resultados esperados sin bajar todavia al nivel de implementacion de codigo.

16.2 Alcance

Describe secuencias funcionales del backend para activacion, lock, resultados, scoring, recalculo y archivado.

16.3 Definiciones

- **Proceso**: secuencia de pasos del sistema para cumplir una regla funcional.
- **Trigger**: evento que inicia un proceso.
- **Recalculo**: ejecucion controlada del scoring y ranking a partir de datos vigentes.

16.4 Reglas

- Cada proceso debe ser trazable y consistente con el dominio.
- Los procesos criticos deben describir triggers, validaciones y resultado esperado.
- Todo proceso critico debe identificar su modulo responsable aunque involucre colaboracion de varios modulos.
- Todo proceso que cambie estados o puntajes debe dejar auditoria suficiente.

16.4.1 Proceso de activacion por comprobante

Modulo responsable: `payments` en colaboracion con `users`

1. Trigger: un usuario registrado carga o presenta un comprobante de pago para revision administrativa.
2. Validaciones principales:
   - el usuario existe y se encuentra en estado `PENDIENTE`;
   - el comprobante es aceptable para revision administrativa;
   - la resolucion administrativa es explicita.
3. Secuencia esperada:
   - se registra el comprobante;
   - se revisa administrativamente;
   - si se aprueba, el usuario transiciona a `ACTIVO`;
   - si se rechaza, el usuario transiciona a `RECHAZADO`.
4. Resultado esperado: el usuario queda en un estado funcional resolutivo y el proceso deja trazabilidad.

16.4.2 Proceso de captura y lock de picks

Modulo responsable: `picks` en colaboracion con `matches`

1. Trigger: un usuario intenta guardar o editar un pick, o el sistema alcanza el umbral de lock por kickoff.
2. Validaciones principales:
   - el usuario esta en estado `ACTIVO`;
   - el partido o prediccion especial sigue en ventana abierta;

- los valores cumplen el rango y formato definido;
- la validacion usa el kickoff oficial vigente del backend.

3. Secuencia esperada:
   - mientras la ventana esta abierta, el pick puede crearse o editarse;
   - al llegar el lock, el sistema bloquea la edicion;
   - si no existe pick guardado, el resultado funcional es `SIN_PICK` sin fabricar una prediccion por defecto.
4. Resultado esperado: ningun pick editable sobrevive despues del lock.

16.4.3 Proceso de sincronizacion de resultados

Modulo responsable: `results`

1. Trigger: ejecucion programada, accion manual de sincronizacion o carga administrativa de resultado.
2. Validaciones principales:
   - el partido existe y puede recibir actualizacion de resultado;
   - la fuente externa o la accion manual produce un valor interpretable por el dominio;
   - si existe override manual vigente, la API automatica no puede sobrescribirlo por si sola.
3. Secuencia esperada:
   - se recibe un resultado externo o manual;
   - el backend lo adapta al contrato interno;
   - se determina la verdad oficial vigente;
   - si hubo cambio efectivo, se registra auditoria y se habilita recalculo.
4. Resultado esperado: cada partido conserva un solo resultado vigente a efectos de scoring.

16.4.4 Proceso de scoring por partido

Modulo responsable: `scoring`

1. Trigger: existe un resultado oficial vigente utilizable para un partido.
2. Validaciones principales:
   - el partido no esta cancelado sin resultado utilizable;
   - los picks asociados son validos o se interpreta correctamente `SIN_PICK`;
   - la fase del partido y su multiplicador oficial son conocidos.
3. Secuencia esperada:
   - se calcula outcome y marcador exacto;
   - se aplica puntaje base;

- se aplica multiplicador con precision exacta;
- se generan o actualizan registros de puntaje reproducibles.

4. Resultado esperado: el puntaje del partido queda determinado de manera unica y reproducible.

16.4.5 Proceso de recalculo de ranking

Modulo responsable: `scoring` en colaboracion con `leaderboard`

1. Trigger: correccion de resultado, override manual, cambio relevante de datos oficiales o accion administrativa de recalc ulo.
2. Validaciones principales:
   - el recalculo parte de resultados vigentes;
   - el proceso es idempotente;
   - los desempates oficiales son aplicados de forma consistente.
3. Secuencia esperada:
   - se identifican partidos o participantes afectados;
   - se recomputa scoring derivado;
   - se reconstruye o actualiza el leaderboard;
   - se registra auditoria del evento y su motivo.
4. Resultado esperado: el ranking resultante puede explicarse desde los datos fuente vigentes.

16.4.6 Proceso de archivado del torneo

Modulo responsable: `admin` en colaboracion con `leaderboard`, `audit` y modulos de dominio

1. Trigger: cierre oficial del torneo y decision administrativa de archivado.
2. Validaciones principales:
   - el torneo ya no requiere operacion competitiva ordinaria;
   - los resultados oficiales vigentes ya fueron consolidados;
   - el archivado se ejecuta como accion explicita y auditada.
3. Secuencia esperada:
   - el torneo pasa a estado `ARCHIVADO`;
   - el sistema queda en modo solo lectura para la competencia cerrada;
   - se preserva evidencia historica para consulta y auditoria.
4. Resultado esperado: ningun flujo ordinario vuelve a abrir la competencia archivada.

16.5 Edge Cases

- Si un kickoff cambia antes del lock, el proceso de lock debe recalcularse con la nueva hora oficial.
- Si un kickoff cambia despues del lock por postergacion, el pick ya bloqueado permanece valido salvo accion administrativa extraordinaria auditada.
- Si un partido se cancela sin resultado utilizable, el proceso de scoring no debe asignar puntos.
- Si un resultado corregido dispara mas de un recalculo, todos los intentos deben converger al mismo estado final.

16.6 Invariantes

- Un proceso critico no puede quedar sin criterio de error ni de auditoria.
- Todo proceso de scoring o recalculo debe partir de una unica verdad oficial vigente por partido.
- Ningun proceso ordinario puede reabrir picks bloqueados ni torneos archivados.

16.7 Preguntas abiertas

- Confirmar si el recalculo sera siempre selectivo por partido afectado o si algunas acciones administrativas permitiran recalculo global del torneo.

16.8 Decisiones pendientes

- 'Decision pendiente' 'No bloqueante': definir si el archivado de torneo incluye snapshots finales obligatorios del leaderboard o si basta con reconstruccion desde datos fuente.
- 'Decision pendiente' 'No bloqueante': confirmar si la sincronizacion de resultados corre solo por scheduler y accion manual o si tambien puede dispararse por eventos adicionales.

17 Validacion y errores

17.1 Descripcion

Este documento define los lineamientos backend para validacion de entrada, manejo de errores e idempotencia. Su objetivo es separar con claridad errores funcionales de errores tecnicos, dejar explicitas las fronteras de validacion y evitar efectos incorrectos en reintentos o recalculos.

17.2 Alcance

Aplica a contratos de entrada, respuestas de error, procesos criticos e interaccion entre validacion, observabilidad y auditoria.

17.3 Definiciones

- **Validacion de frontera**: verificacion de entradas en el limite del sistema.
- **Idempotencia**: repeticion segura de una operacion sin efectos incorrectos adicionales.

- **Error funcional**: error derivado de una regla del dominio o de una accion no permitida.
- **Error tecnico**: error derivado de infraestructura, integracion, persistencia o fallas de ejecucion.

17.4 Reglas

- Toda entrada backend debe validarse en la frontera.
- Los errores funcionales deben separarse de errores tecnicos.
- Recalculo y sincronizacion deben describirse con criterio de idempotencia.
- Ningun mensaje de error debe exponer detalles internos innecesarios al consumidor del API.

17.4.1 Validacion de frontera

- Toda solicitud entrante debe validarse antes de ejecutar logica de dominio.
- La validacion de frontera debe cubrir forma, tipos, rangos, obligatoriedad y consistencia basica del contrato.
- Las reglas del dominio se validan despues de la frontera, dentro de la capa de aplicacion o dominio segun corresponda.
- La validacion de frontend mejora usabilidad, pero no reemplaza la validacion oficial del backend.

Ejemplos de validacion de frontera esperada:

- rango de goles de 0 a 9;
- formato valido de identificadores;
- campos obligatorios presentes;
- estructura valida de payloads administrativos;
- tipos correctos para fechas, puntajes y estados.

17.4.2 Taxonomia de errores

Errores funcionales esperados:

- usuario no autorizado para la accion;
- usuario no activo intentando guardar picks;
- intento de editar pick despues del lock;
- resultado o partido en estado incompatible con la accion solicitada;
- payload valido en forma, pero incompatible con una regla del dominio.

Errores tecnicos esperados:

- falla de base de datos;
- timeout o error del proveedor externo;
- error inesperado de proceso o serializacion;

- imposibilidad de persistir auditoria o logs operativos.

Reglas de respuesta:

- Un error funcional debe comunicar que regla impide la accion sin filtrar detalles internos de implementacion.
- Un error tecnico debe responder de forma segura, con mensaje estable para cliente y detalle tecnico en logs.
- La misma condicion debe producir la misma categoria de error de forma consistente entre endpoints equivalentes.

17.4.3 Idempotencia

- La sincronizacion de resultados debe ser idempotente cuando reciba repetidamente el mismo valor oficial vigente.
- El recalculo de scoring debe ser idempotente frente a reintentos sobre el mismo conjunto de datos fuente.
- El archivado del torneo no debe producir multiples estados finales incompatibles si el comando se repite.
- Una correccion manual de resultado debe dejar un solo valor vigente aunque la accion administrativa se procese mas de una vez por error operativo.

17.4.4 Relacion entre error, log y auditoria

- Todo error tecnico relevante debe quedar en log estructurado.
- Toda accion funcional sensible, incluso si falla parcialmente, debe evaluarse para dejar rastro auditable cuando aplique.
- Logs tecnicos y auditoria funcional son complementarios: los logs explican diagnostico tecnico y la auditoria explica el efecto funcional.
- Ningun error critico debe impedir el intento de registrar contexto suficiente para diagnostico.

17.5 Edge Cases

- Si una solicitud pasa la validacion de forma pero falla por regla de dominio, debe responderse como error funcional y no como error tecnico.
- Si un proceso automatico falla despues de un cambio parcial, el sistema debe reflejar el estado real alcanzado y dejar rastro operativo del fallo.
- Si la API externa entrega un valor repetido, la sincronizacion no debe duplicar efectos ni producir nuevas auditorias innecesarias.
- Si la auditoria no puede persistirse durante una accion critica, el sistema debe tratarlo como incidente operativo relevante.

17.6 Invariantes

- Ningun error critico debe impedir auditoria de acciones relevantes.
- Ninguna entrada llega al dominio sin pasar por validacion de frontera.
- Ningun reintento legitimo debe duplicar scoring ni corromper el estado oficial.
- Ninguna respuesta al cliente debe exponer secretos, stack traces ni detalles internos sensibles.

17.7 Preguntas abiertas

- Confirmar si el contrato conceptual del backend documentara explicitamente una taxonomia de codigos de error estable por modulo.

17.8 Decisiones pendientes

- 'Decision pendiente' 'No bloqueante': definir si cada modulo expondra codigos de error funcional propios o si existira una taxonomia comun transversal.
- 'Decision pendiente' 'No bloqueante': confirmar que operaciones administrativas exigiran persistencia obligatoria de auditoria antes de considerar exitosa la accion.

18 Endpoints conceptuales

18.1 Descripcion

Este documento define el catalogo conceptual de endpoints y operaciones esperadas del backend. Su objetivo es describir que capacidades debe exponer el API para soportar frontend y operacion administrativa, sin caer todavia en una especificacion exhaustiva tipo OpenAPI.

18.2 Alcance

La intencion es descriptiva. No reemplaza una especificacion exhaustiva tipo OpenAPI ni fija detalles finales de verbos HTTP, codigos de estado o esquemas fisicos.

18.3 Definiciones

- **Endpoint conceptual**: operacion esperada del backend expresada a nivel funcional.
- **Operacion administrativa**: endpoint reservado a acciones autorizadas del panel de administracion.

18.4 Reglas

- El documento describira proposito, entrada esperada y salida esperada por operacion.
- Los endpoints no podran contradecir reglas del dominio ni limites arquitectonicos.
- Toda operacion conceptual debe pertenecer claramente a un modulo responsable.
- El frontend consume contratos de dominio estables, no detalles internos de persistencia o integracion.

18.4.1 Operaciones publicas y autenticadas esperadas

Modulo	Operacion conceptual	Entrada esperada	Salida esperada
`auth`	iniciar sesion	credenciales validas	sesion o token utilizable por frontend
`auth`	recuperar acceso	identificador del usuario o correo	confirmacion del flujo de recuperacion
`users`	registrar usuario	datos de registro	usuario creado en estado `PENDIENTE`
`users`	consultar perfil propio	sesion valida	datos de cuenta, estado y contexto del usuario
`payments`	registrar comprobante	referencia o evidencia de pago	comprobante recibido para revision
`matches`	listar partidos visibles	filtros opcionales por fase o jornada	calendario visible con estado y kickoff
`picks`	guardar pick por partido	partido, goles local y visita	pick persistido o rechazo por regla funcional
`picks`	consultar picks propios	sesion valida, filtros opcionales	picks propios y su estado funcional
`picks`	guardar prediccion especial	tipo de prediccion y valor elegido	prediccion especial persistida o rechazo por lock
`leaderboard`	consultar leaderboard general	filtros opcionales	ranking general con puntaje acumulado
`leaderboard`	consultar leaderboard por fase o jornada	identificador de fase o jornada	vista filtrada del ranking
`results`	consultar resultado visible de partido	identificador del partido	resultado oficial vigente visible para frontend

18.4.2 Operaciones administrativas esperadas

Modulo	Operacion conceptual	Entrada esperada	Salida esperada
`payments`	aprobar comprobante	identificador de comprobante y resolucion	usuario activado y evento auditado
`payments`	rechazar comprobante	identificador de comprobante y motivo	usuario rechazado y evento auditado
`users`	bloquear usuario	identificador de usuario y motivo	cuenta bloqueada y trazabilidad registrada
`matches`	actualizar kickoff o estado de partido	identificador de partido y nuevo valor	partido actualizado con auditoria
`results`	cargar o corregir resultado manualmente	identificador de partido, valor y justificacion	resultado vigente actualizado y recalculo habilitado
`scoring`	ejecutar recalculo	alcance del recalculo y motivo	scoring y ranking recalculados

`admin`	archivar torneo	identificador de torneo y confirmacion autorizada	torneo archivado en modo solo lectura
`audit`	consultar eventos auditables	filtros administrativos	eventos auditables visibles para operacion

18.4.3 Reglas de contrato conceptual

- Ninguna operacion de picks debe asumir que el frontend es autoridad de lock; el backend siempre revalida.
- Las operaciones de resultados deben devolver solo la verdad oficial vigente y no exponer al frontend la complejidad interna del proveedor externo.
- Las operaciones administrativas deben exigir autorizacion, justificacion cuando aplique y salida trazable.
- Las vistas de leaderboard deben responder con el ranking derivado oficial, no con datos parciales sin contexto de scoring.

18.5 Edge Cases

- Una operacion conceptualmente valida puede terminar rechazada por estado funcional del usuario, lock o falta de permisos.
- Una operacion de sincronizacion o correccion de resultado puede disparar recalculo sin que el frontend invoque ese proceso de forma directa.
- Si un partido fue cancelado sin resultado utilizable, las operaciones de scoring derivado no deben producir puntaje asociado.

18.6 Invariantes

- Los contratos conceptuales deben mantener consistencia entre frontend y backend.
- Ningun endpoint conceptual redefine reglas oficiales ya cerradas en dominio.
- Ninguna operacion administrativa critica existe sin posibilidad de auditoria.

18.7 Preguntas abiertas

- Confirmar si el backend expondra operaciones separadas para predicciones especiales o si esas acciones viviran bajo un mismo grupo de picks con distincion por tipo.

18.8 Decisiones pendientes

- ‘Decision pendiente‘ ‘No bloqueante‘: definir si el contrato conceptual documentara nombres de rutas y verbos HTTP en una iteracion posterior o si eso quedara fuera de este repositorio.
- ‘Decision pendiente‘ ‘No bloqueante‘: confirmar si la consulta administrativa de auditoria se expone como endpoint propio del modulo `audit` o como capacidad interna del panel admin.

19 Estructura por features

19.1 Descripcion

Este documento define la estructura funcional esperada del repositorio frontend para v1. Su objetivo es dejar claro como se organiza el trabajo por features, que responsabilidad tiene cada bloque del arbol del frontend y que limites deben respetarse para no mezclar UI, logica de flujo, contratos y piezas compartidas.

19.2 Alcance

Aplica al repositorio frontend de la plataforma Quiniela. Describe una estructura objetivo para trabajo diario, mantenimiento y crecimiento controlado del producto. No define detalle visual ni decisiones finas de implementacion de componentes.

19.3 Definiciones

- **Feature**: unidad funcional del frontend asociada a un objetivo claro del usuario o del administrador.
- **Shared**: espacio comun reservado para piezas reutilizables que no dependen de un flujo especifico del negocio.
- **Estado remoto**: datos obtenidos del backend y sincronizados en cliente.
- **Estado local**: estado efimero de interfaz que no representa la verdad persistente del dominio.
- **Ownership funcional**: responsabilidad principal de una feature o bloque sobre sus pantallas, validaciones de experiencia, consultas y componentes propios.

19.4 Reglas

- El frontend se organizara principalmente por features y no por tipo tecnico aislado de archivo.
- Cada feature debe responder a un flujo reconocible del producto y tener ownership claro.
- La estructura del frontend no debe ocultar reglas criticas del dominio dentro de componentes visuales o utilidades compartidas.
- El frontend puede validar por experiencia de usuario, pero no redefine locks, scoring, permisos ni resultados oficiales.
- `shared` solo puede contener piezas autenticamente reutilizables y neutrales respecto del dominio especifico.
- Las features deben depender de contratos del backend ya definidos, no inventar modelos paralelos incompatibles.

19.4.1 Features objetivo de v1

La estructura esperada para v1 parte del siguiente inventario funcional:

- `auth`: login, recuperacion de contrasena y continuidad de sesion.

- `activation`: estado de activacion, instrucciones de pago y comprobante.
- `picks`: captura, edicion y consulta de picks por partido.
- `special-predictions`: captura y consulta de `Campeon` y `Goleador`.
- `leaderboard`: tabla general y vistas derivadas de ranking.
- `profile`: datos propios del participante y estado de su cuenta.
- `admin`: operaciones administrativas permitidas para usuarios con privilegios.

19.4.2 Tree funcional sugerido

El siguiente tree es una referencia descriptiva para el repositorio frontend:

```
frontend/
|-- app/
|   |-- rutas y layouts
|   |-- providers globales
|   `-- boundaries de navegacion
|-- features/
|   |-- auth/
|   |-- activation/
|   |-- picks/
|   |-- special-predictions/
|   |-- leaderboard/
|   |-- profile/
|   `-- admin/
|-- entities/
|   |-- user/
|   |-- match/
|   |-- pick/
|   |-- special-prediction/
|   |-- result/
|   `-- leaderboard-entry/
`-- shared/
    |-- ui/
    |-- lib/
    |-- config/
    `-- contracts/
```

19.4.3 Responsabilidad por bloque

- `app`: define composicion global, rutas, layouts y configuracion de alto nivel del frontend.
- `features`: concentra comportamiento orientado a flujos concretos del producto.
- `entities`: concentra modelos de presentacion y adaptadores reutilizables por multiples features cuando eso reduzca duplicacion real.
- `shared`: contiene primitivas visuales, utilidades generales, configuracion comun y contratos neutrales.

19.4.4 Reglas de ownership por feature

- `auth` es duena de formularios de acceso, recuperacion de contrasena y control de sesion visible al usuario.
- `activation` es duena del flujo de comprobante, estados visibles de activacion y mensajes relacionados con habilitacion de cuenta.
- `picks` es duena de la captura y consulta de picks por partido, incluyendo estados de editable, bloqueado y sin pick.
- `special-predictions` es duena del flujo separado de `Campeon` y `Goleador` dentro del modulo funcional de picks.
- `leaderboard` es duena de la visualizacion del ranking y del desglose que el producto decida exponer.
- `profile` es duena de la informacion propia del participante no administrativa.
- `admin` es duena de operaciones privilegiadas, vistas administrativas y acciones auditables expuestas en frontend.

19.4.5 Reglas para shared

- Una pieza solo entra en `shared` si puede reutilizarse en mas de una feature sin transportar reglas especificas de negocio.
- Un componente visual generico puede vivir en `shared/ui` si no codifica comportamiento propio de picks, scoring, activacion o administracion.
- Una validacion ligada al dominio de picks, resultados o scoring no pertenece a `shared`; pertenece al feature o entidad correspondiente.
- Los contratos de red compartidos pueden vivir en `shared/contracts` si representan definiciones comunes consumidas por varias features.

19.4.6 Reglas de dependencia

- `app` puede componer features, entities y piezas compartidas.
- Una feature puede depender de entities y shared, pero no debe depender de otra feature salvo composicion muy controlada y justificada.
- `shared` no debe depender de features.
- Ninguna capa del frontend accede directamente a la base de datos ni a la API externa de resultados.
- El frontend depende del backend como fuente de verdad para locks, permisos, resultados oficiales y puntajes.

19.5 Edge Cases

- Si una feature comparte demasiada logica con otra, debe revisarse si existe una entidad comun reusable o si la separacion actual esta mal planteada.

- Si una pieza ubicada en `shared` requiere conocimiento especifico de scoring, picks o activacion, debe salir de `shared`.
- Si una validacion en cliente mejora la experiencia, backend sigue siendo la autoridad final para aceptar o rechazar la operacion.

19.6 Invariantes

- La estructura del frontend no redefine reglas oficiales cerradas en `01_Producto_y_Reglas`.
- Cada feature debe mapear a un flujo reconocible del producto y no a una agrupacion arbitraria de archivos.
- El estado remoto y el estado local deben mantenerse como preocupaciones distintas.
- El frontend no debe asumir que un lock, permiso o puntaje es valido sin confirmacion del backend.

19.7 Preguntas abiertas

- Confirmar si `special-predictions` quedara como feature independiente en el repositorio frontend o como subestructura interna de `picks`.
- Confirmar si la capa `entities` es necesaria desde v1 o si el volumen real del frontend permite resolver esa responsabilidad dentro de `features` y `shared`.

19.8 Decisiones pendientes

- `Decision pendiente No bloqueante`: confirmar el inventario visual final de pantallas por feature para aterrizar la estructura definitiva de carpetas del frontend.
- `Decision pendiente No bloqueante`: decidir si la capa `entities` vivira como bloque independiente desde el inicio o se introducira solo si la complejidad del repositorio lo justifica.

20 Flujos y paginas

20.1 Descripcion

Este documento define los flujos principales y las superficies de pagina esperadas del frontend para v1. Su objetivo es describir que recorrido funcional debe existir para usuarios publicos, participantes autenticados y administradores, sin entrar todavia al detalle visual fino de UI.

20.2 Alcance

Aplica a rutas, paginas, estados visibles y accion principal de cada flujo del frontend. No define microcopy exhaustivo ni detalle final de componentes.

20.3 Definiciones

- **Flujo**: recorrido funcional completo para lograr un objetivo del producto.
- **Pagina**: superficie principal de navegacion con una accion o proposito dominante.

- **Estado visible**: condicion relevante que la pantalla debe expresar al usuario, aunque la validacion final pertenezca al backend.

20.4 Reglas

- Cada flujo debe alinearse con reglas cerradas del dominio y con los contratos conceptuales del backend.
- Cada pagina debe tener una accion principal reconocible y evitar mezclar demasiados objetivos funcionales.
- El frontend no debe habilitar recorridos que contradigan estados funcionales del usuario, partido, pick o torneo.
- Cuando una accion dependa de locks, permisos o estado de activacion, la pagina debe reflejar la restriccion de forma visible antes y despues del rechazo del backend.

20.4.1 Inventario funcional de paginas esperadas

Pagina o vista	Tipo de acceso	Accion principal
Inicio / landing	publico	explicar el producto y dirigir a registro o acceso
Registro	publico	crear cuenta nueva
Login	publico	iniciar sesion
Recuperacion de contrasena	publico	iniciar flujo de recuperacion
Estado de activacion	autenticado	informar estado de cuenta y siguientes pasos
Carga de comprobante	autenticado	registrar comprobante para revision
Calendario y picks	autenticado	consultar partidos y capturar picks por partido
Predicciones especiales	autenticado	capturar o consultar `Campeon` y `Goleador`
Leaderboard	autenticado	consultar ranking general y vistas filtradas
Perfil	autenticado	consultar contexto propio de cuenta
Panel admin de usuarios y pagos	admin	revisar usuarios, comprobantes y resoluciones
Panel admin de partidos y resultados	admin	actualizar kickoff, estados y resultados
Panel admin de scoring y auditoria	admin	disparar recalculos y consultar trazabilidad

20.4.2 Flujos principales de participante

Flujo de registro y acceso

- El usuario publico debe poder registrarse, iniciar sesion y recuperar acceso.
- Despues del registro, la primera vista funcional relevante debe dejar claro que la cuenta permanece en `PENDIENTE` hasta resolucion administrativa.
- Si el usuario esta autenticado pero no activado, el frontend debe priorizar el flujo de activacion sobre flujos de juego.

Flujo de activacion

- El usuario en `PENDIENTE` debe poder consultar instrucciones y registrar comprobante.
- La pantalla debe reflejar claramente si la cuenta sigue `PENDIENTE`, fue `RECHAZADA` o ya esta `ACTIVA`.
- El frontend no debe inventar un estado `EN_REVISION`; la revision administrativa vive dentro del flujo, no como estado funcional separado del usuario.

Flujo de picks por partido

- Solo un usuario `ACTIVO` puede capturar o editar picks por partido.
- La pantalla de picks debe mostrar el estado funcional del partido y del pick: editable, bloqueado o ausencia valida de pick.
- Si el partido ya esta bloqueado, el frontend debe mostrar modo lectura y no presentar controles de edicion activos.
- Si no existe prediccion guardada al momento del lock, la vista debe comunicar que el partido quedo sin pick y con cero puntos potenciales, sin fabricar valores por defecto.

Flujo de predicciones especiales

- `Campeon` y `Goleador` deben vivir en un flujo separado dentro del modulo funcional de picks.
- Ese flujo debe reflejar su propio cierre: cinco minutos antes del kickoff oficial del primer partido del torneo.
- Una vez cerradas, las predicciones especiales deben pasar a modo lectura para el participante.

Flujo de leaderboard

- El leaderboard debe mostrar el puntaje total acumulado como criterio principal de orden.
- El frontend debe poder ofrecer desglose de aportes por picks y predicciones especiales si el producto decide exponerlo.
- La consulta del ranking no debe recalcular logica en cliente; solo presenta el derivado oficial recibido del backend.

20.4.3 Flujos principales de administracion

- El panel administrativo debe separarse funcionalmente de la experiencia de participante.
- Deben existir vistas claras para:

  - usuarios y comprobantes;
  - partidos y resultados;
  - scoring, recalculo y auditoria.
- Las acciones criticas del admin deben exigir confirmacion visible y dejar contexto suficiente para una operacion segura.

20.4.4 Estados visibles minimos por pagina

- Carga: mientras se espera informacion remota.
- Vacio: cuando no existen datos relevantes para mostrar.
- Error recuperable: cuando la operacion puede reintentarse o corregirse.
- Restringido: cuando el estado funcional o el permiso no permite la accion.
- Solo lectura: cuando la informacion es visible pero ya no editable.

20.5 Edge Cases

- Un usuario autenticado pero `PENDIENTE`, `RECHAZADO` o `BLOQUEADO` puede acceder a partes visibles del frontend, pero no debe poder ejecutar flujos reservados a `ACTIVO`.
- Un partido `POSTERGADO` puede requerir actualizar la presentacion del lock sin que eso implique permitir al frontend decidir la regla por si mismo.
- Un resultado `CORREGIDO` puede requerir aviso visible al usuario sin exponer detalle operativo completo del cambio.

20.6 Invariantes

- Ningun flujo del frontend puede permitir editar picks despues del lock oficial.
- Ninguna pagina administrativa debe estar disponible sin autorizacion valida.
- Las pantallas del frontend representan estados del dominio; no los redefinen.
- Toda pagina con accion critica debe dejar claro su estado de exito, rechazo o restriccion.

20.7 Preguntas abiertas

- Confirmar si el leaderboard por jornada sera una pantalla propia o un filtro dentro de una unica superficie de ranking.
- Confirmar si el aviso de resultado corregido aparecera en calendario, detalle de partido, leaderboard o en mas de una superficie del frontend.

20.8 Decisiones pendientes

- `Decision pendiente No bloqueante`: decidir el inventario final de paginas administrativas de auditoria para v1 y si esa vista vive dentro del panel admin o como subseccion especializada.

- `Decision pendiente No bloqueante`: confirmar si la landing publica de v1 requiere contenido editorial extendido o si basta una superficie funcional enfocada en registro y acceso.

21 Estado, validacion y datos

21.1 Descripcion

Este documento define como debe manejar el frontend el estado remoto, el estado local de interfaz, la validacion de entradas y el consumo de datos provenientes del backend. Su objetivo es evitar que el cliente mezcle preocupaciones, replique reglas oficiales de negocio o construya contratos ambiguos.

21.2 Alcance

Aplica al repositorio frontend en su integracion con backend. Describe principios y reglas de manejo de datos; no reemplaza la documentacion de contratos conceptuales del backend ni el detalle de implementacion de librerias.

21.3 Definiciones

- **Estado remoto**: informacion obtenida del backend y considerada verdad vigente para el frontend.
- **Estado local**: estado efimero de interfaz usado para composicion visual, formularios y comportamiento interactivo no persistido.
- **Validacion de frontera**: validacion realizada en el punto de entrada o salida de datos entre frontend y backend.
- **Contrato**: estructura esperada de entrada o salida consumida por el frontend.

21.4 Reglas

- El frontend debe tratar el estado remoto y el estado local como preocupaciones separadas.
- Los datos de dominio visibles en el cliente deben venir del backend y consumirse bajo contratos explicitamente validados.
- La validacion del frontend existe para mejorar experiencia y reducir errores de captura, no para sustituir enforcement del backend.
- El frontend no debe asumir como verdadero ningun lock, permiso, resultado o puntaje que no haya sido confirmado por el backend.
- Los errores funcionales y los errores tecnicos deben mostrarse de manera diferenciada en la experiencia de usuario.

21.4.1 Manejo de estado remoto

- El estado remoto incluye, al menos, sesion, perfil, estado de activacion, calendario de partidos, picks propios, predicciones especiales, leaderboard y resultados visibles.

- El frontend debe considerar como fuente de verdad vigente el dato mas reciente validamente recibido del backend.
- Los datos remotos deben poder invalidarse o refrescarse cuando una mutacion cambie el estado funcional del sistema.
- Las vistas derivadas del frontend no deben persistirse como verdad paralela si pueden obtenerse de respuestas oficiales del backend.

21.4.2 Manejo de estado local

- El estado local se limita a composicion visual, estado temporal de formularios, controles abiertos o cerrados, filtros no persistidos y feedback inmediato de interaccion.
- Un dato local no puede usarse como sustituto durable de una respuesta oficial del backend.
- Cuando una accion critica depende de informacion remota, el frontend debe reconciliar su estado local con la respuesta del servidor despues de la operacion.

21.4.3 Reglas de validacion

- El frontend debe validar entradas visibles del usuario antes de enviar la solicitud cuando eso mejore claridad o reduzca errores evitables.
- La validacion de picks debe respetar que los goles permitidos son enteros entre `0` y `9`.
- La validacion de predicciones especiales debe respetar su cierre propio y el formato esperado de seleccion.
- La validacion de formularios no debe ocultar errores devueltos por backend; debe integrarlos y mostrarlos de forma comprensible.
- Si una operacion es rechazada por lock, permiso o estado funcional, el frontend debe privilegiar el mensaje funcional sobre el detalle tecnico.

21.4.4 Contratos y acceso a datos

- El frontend consumira contratos conceptuales estables del backend y adaptara sus respuestas a modelos de presentacion cuando sea necesario.
- Las adaptaciones de contrato deben vivir cerca del acceso a datos o de la entidad correspondiente, no dispersas en componentes visuales.
- El frontend no debe construir contratos paralelos incompatibles con backend para simplificar localmente una pantalla.
- Toda mutacion relevante debe definir que consultas deben refrescarse o invalidarse despues de completarse.

21.4.5 Datos minimos por feature

Feature	Datos remotos principales	Estado local frecuente

`auth`	sesion y contexto basico de cuenta	estado de formulario y feedback de acceso
`activation`	estado de usuario, comprobante e instrucciones visibles	carga de archivo, confirmacion y mensajes de revision
`picks`	partidos, kickoff, picks propios y estado funcional del partido	valores editables, filtros y confirmaciones de guardado
`special-predictions`	predicciones especiales propias y su estado de cierre	seleccion temporal antes de guardar
`leaderboard`	ranking general, filtros y desglose disponible	filtros activos y preferencia de vista
`profile`	datos propios del participante	expansion de secciones y edicion local si aplica
`admin`	usuarios, comprobantes, partidos, resultados, recalculos y auditoria visible	filtros, confirmaciones y contexto de accion critica

21.4.6 Reglas de error y feedback

- Un error funcional debe explicar que regla impidio la accion, por ejemplo lock vencido, cuenta no activa o falta de permisos.
- Un error tecnico debe comunicar que la operacion no pudo completarse y ofrecer reintento cuando corresponda.
- El frontend no debe esconder el estado final de una mutacion critica; siempre debe dejar feedback de exito, rechazo o fallo tecnico.
- Cuando exista resultado corregido, el frontend debe poder mostrar un aviso explicito y breve sin reconstruir la auditoria interna.

21.5 Edge Cases

- Si el frontend muestra un formulario editable y el backend rechaza por lock reciente, la vista debe reconciliarse con el nuevo estado funcional y pasar a modo lectura.
- Si un dato remoto cambia por recalculo o correccion, las vistas derivadas deben invalidarse para evitar mostrar ranking o puntajes obsoletos.
- Si una consulta administrativa retorna informacion parcial por permisos, el frontend debe degradar la vista de forma controlada y no asumir acceso total.

21.6 Invariantes

- El backend sigue siendo la autoridad final para validar y persistir operaciones del dominio.
- Ningun componente del frontend debe depender de datos no garantizados por contrato.
- La logica de presentacion no debe contener reglas oficiales unicas de scoring, lock o permisos.
- Toda mutacion critica debe tener un camino claro de refresco o invalidacion del estado remoto afectado.

21.7 Preguntas abiertas

- Confirmar si los esquemas Zod de frontend se mantendran totalmente independientes o si existira una estrategia posterior de comparticion controlada de contratos con backend.
- Confirmar cuantas superficies del frontend mostraran desglose de puntaje y con que precision visual se representaran los decimales.

21.8 Decisiones pendientes

- `Decision pendiente No bloqueante`: definir la politica exacta de invalidacion y refresco para leaderboard y picks despues de una correccion de resultado.
- `Decision pendiente No bloqueante`: decidir si los mensajes funcionales del frontend tendran catalogo centralizado por feature o un inventario transversal de microcopy operativo.

22 Modelo de dominio

22.1 Descripcion

Este documento presenta el dominio funcional del sistema y sus agrupaciones conceptuales principales. Su objetivo es expresar el lenguaje del negocio antes de cualquier detalle de tablas, endpoints o decisiones fisicas de persistencia.

22.2 Alcance

Aplica al dominio central de la plataforma de quiniela.

22.3 Definiciones

- **Dominio**: conjunto de reglas y conceptos que definen como funciona la quiniela.
- **Subdominio**: area funcional con responsabilidad delimitada dentro del sistema.

22.4 Reglas

Subdominios principales:

- ‘Identidad y acceso‘: registro, autenticacion y estado de usuario.
- ‘Activacion y pagos‘: comprobante, revision administrativa y habilitacion posterior del usuario.
- ‘Torneo y calendario‘: torneo, equipos, partidos, fases y kickoff.
- ‘Picks‘: predicciones por partido y predicciones especiales de torneo.
- ‘Resultados‘: recepcion de resultados oficiales y correcciones.
- ‘Scoring‘: calculo de puntos, multiplicadores y desempates.
- ‘Leaderboard‘: posicionamiento general y filtrado.
- ‘Administracion‘: acciones operativas de la JD.
- ‘Auditoria‘: trazabilidad de acciones y cambios.

- ‘Notificaciones‘: eventos comunicables por correo.

Reglas de separacion conceptual:

- El estado funcional del usuario pertenece a ‘Identidad y acceso‘.
- La revision del comprobante pertenece a ‘Activacion y pagos‘ y no crea por si misma un nuevo estado de usuario distinto de `PENDIENTE`.
- Las predicciones especiales pertenecen al subdominio ‘Picks‘, aunque su ventana de captura y su evaluacion sean distintas de los picks por partido.
- ‘Resultados‘ mantiene una unica verdad vigente por partido para efectos de scoring, aunque pueda existir historia de correcciones y overrides.
- ‘Leaderboard‘ es un subdominio derivado: no define reglas propias de juego, solo expresa el resultado acumulado del scoring y sus desempates.

Relaciones de dominio:

- Un usuario participa en un torneo mediante picks por partido y predicciones especiales.
- Un comprobante de pago puede sustentar la activacion manual de un usuario, pero no equivale por si mismo a habilitacion competitiva.
- Un partido pertenece a una fase del torneo y usa un kickoff oficial vigente para su lock.
- Un resultado oficial vigente alimenta el scoring de todos los picks asociados al partido.
- El scoring consolida puntaje por partido y puntaje de predicciones especiales para actualizar el leaderboard.
- Acciones administrativas, correcciones y recalculos generan auditoria.

22.5 Edge Cases

- El dominio distingue entre resultado oficial y override manual, pero ambos deben converger a una sola verdad vigente.
- El dominio debe soportar ausencia de pick sin convertirla en un pick implicito.
- El dominio debe soportar partidos postergados, suspendidos, cancelados o corregidos sin romper la trazabilidad de scoring.

22.6 Invariantes

- Las reglas del juego pertenecen al dominio y no a UI ni a integraciones.
- Un resultado oficial vigente debe ser unico por partido a efectos de scoring.
- Ninguna correccion de resultado puede eliminar la historia operativa previa.
- La ausencia de pick es un resultado funcional valido del dominio, no un valor por defecto fabricado por el sistema.

22.7 Preguntas abiertas

- Confirmar si el modelo conceptual necesita distinguir explicitamente entre puntaje calculado y snapshot persistido de leaderboard cuando se documente el modelo derivado.

23 Entidades y relaciones

23.1 Descripcion

Este documento enumera las entidades conceptuales del sistema y sus relaciones principales. El nivel es funcional y no pretende reemplazar un esquema fisico de base de datos.

23.2 Alcance

Aplica al modelo conceptual de datos de la plataforma.

23.3 Definiciones

- **Entidad**: objeto de dominio con identidad propia.
- **Relacion**: vinculo funcional entre entidades.

23.4 Reglas

Entidades principales:

Entidad	Responsabilidad funcional
User	Identidad del participante o administrador.
PaymentProof	Evidencia de pago usada para activacion manual.
Tournament	Contenedor del torneo documentado por la plataforma.
Phase	Segmentacion oficial del torneo para scoring y vistas.
Team	Equipo participante del torneo.
Match	Partido con kickoff, fase y estado operativo.
Pick	Prediccion de marcador exacto por usuario y partido.
SpecialPrediction	Prediccion especial de Campeon o Goleador dentro del subdominio de picks.
Result	Resultado oficial o corregido de un partido.
ScoreEntry	Registro derivado del puntaje aplicado a un pick.
LeaderboardSnapshot	Vista derivada opcional del ranking en un momento dado.
AuditEvent	Evento auditable generado por accion critica o correccion.

Notas de modelado conceptual:

- ‘Pick‘ y ‘SpecialPrediction‘ se documentan como conceptos distintos porque tienen distinta cardinalidad, distinta ventana de lock y distinta forma de evaluacion.
- Esta distincion conceptual no obliga por si misma a una implementacion fisica especifica en base de datos.

- ‘LeaderboardSnapshot‘ se mantiene como entidad derivada opcional; la fuente normativa del ranking sigue siendo el scoring reproducible.

Relaciones principales:

- ‘User‘ genera ‘PaymentProof‘.
- ‘User‘ crea ‘Pick‘ y ‘SpecialPrediction‘.
- ‘Tournament‘ contiene ‘Phase‘, ‘Team‘ y ‘Match‘.
- ‘Tournament‘ define tambien el contexto unico de cada ‘SpecialPrediction‘.
- ‘Match‘ tiene cero o un ‘Result‘ vigente.
- ‘Pick‘ se evalua contra ‘Result‘ para producir ‘ScoreEntry‘.
- ‘SpecialPrediction‘ se evalua contra el cierre oficial del torneo o contra su estadistica oficial correspondiente para producir puntaje acumulable.
- ‘AuditEvent‘ puede referenciar acciones sobre usuarios, pagos, partidos, resultados y recalculos.

23.5 Edge Cases

- Un ‘Match‘ puede pasar por estados no lineales como postergado o corregido; el modelo debe preservar historia suficiente para auditar.
- Un ‘User‘ puede permanecer en `PENDIENTE` mientras su ‘PaymentProof‘ atraviesa revision administrativa sin que eso implique nuevos estados funcionales del usuario.
- ‘LeaderboardSnapshot‘ es opcional; el ranking puede derivarse o persistirse segun necesidad operativa.

23.6 Invariantes

- Un ‘Pick‘ pertenece a un solo ‘User‘ y a un solo ‘Match‘.
- Una ‘SpecialPrediction‘ pertenece a un solo ‘User‘, a un solo ‘Tournament‘ y a un solo tipo de prediccion.
- Un ‘Result‘ vigente a efectos de scoring debe ser unico por ‘Match‘.
- Cada ‘AuditEvent‘ debe poder asociarse a un actor o a un proceso identificable.

23.7 Preguntas abiertas

- Confirmar si ‘ScoreEntry‘ registrara tambien de forma explicita el puntaje de ‘SpecialPrediction‘ o si ese puntaje vivira en una estructura derivada distinta.

24 Invariantes de datos

24.1 Descripcion

Este documento concentra las condiciones que siempre deben cumplirse en el modelo de datos y en la persistencia funcional del sistema.

24.2 Alcance

Aplica a la consistencia logica de usuarios, partidos, picks, resultados, scoring y auditoria.

24.3 Definiciones

- **Invariante**: regla que debe mantenerse verdadera en todo momento valido del sistema.

24.4 Reglas

- Un usuario no activo no puede registrar picks validos.
- No puede existir mas de un pick vigente por usuario y partido.
- No puede existir mas de una prediccion especial vigente por usuario, torneo y tipo de prediccion.
- Un pick bloqueado no puede mutar.
- Un partido tiene un solo kickoff oficial vigente a la vez.
- Un resultado vigente por partido debe ser unico.
- El scoring debe poder recalcularse sin duplicar puntajes.
- Un partido cancelado no produce puntaje.
- La ausencia de pick antes del lock no puede persistirse como un pick fabricado con valores por defecto.
- Toda correccion de resultado debe preservar referencia al valor anterior mediante auditoria.
- El ranking debe ser reproducible a partir de resultados oficiales y reglas de desempate.

24.5 Edge Cases

- Si se persisten snapshots del leaderboard, estos no reemplazan la posibilidad de reconstruccion a partir de datos fuente.
- Si un usuario es bloqueado despues de haber jugado, sus datos historicos se conservan para trazabilidad salvo una politica explicita distinta.
- Si existe un override manual vigente sobre un resultado, ninguna sincronizacion automatica debe reemplazarlo sin una nueva accion explicita y auditada.

24.6 Invariantes

- Ninguna entidad critica del dominio puede quedar sin identificacion suficiente para auditoria.
- Toda operacion administrativa sensible debe dejar evidencia trazable.

- Todo ranking publicado debe poder explicarse desde resultados vigentes, scoring reproducible y desempates oficiales.

24.7 Preguntas abiertas

- Confirmar politica de conservacion de datos para usuarios bloqueados o rechazados.

24.8 Decisiones pendientes

- ‘Decision pendiente‘ ‘No bloqueante‘: definir estrategia de retencion historica de snapshots de leaderboard.

25 Auditoria y trazabilidad

25.1 Descripcion

Este documento define que eventos deben quedar auditados y que informacion minima necesita cada registro para soportar operacion, revision y resolucion de disputas.

25.2 Alcance

Aplica a acciones administrativas, cambios de estado relevantes, overrides y procesos automaticos sensibles.

25.3 Definiciones

- **Evento auditable**: accion o proceso cuya ocurrencia debe poder demostrarse despues.
- **Actor**: usuario o proceso responsable de un evento.

25.4 Reglas

Eventos que deben auditarse como minimo:

- aprobacion o rechazo de usuario;
- bloqueo de usuario;
- carga, revision y validacion de comprobante;
- cambio de kickoff;
- ingreso manual o correccion de resultado;
- recalculo de puntajes;
- archivado de torneo;
- errores operativos relevantes de integracion de resultados.

Campos minimos del evento auditable:

- identificador del evento;
- fecha y hora;

- actor;
- tipo de accion;
- entidad afectada;
- valor anterior cuando aplique;
- valor nuevo cuando aplique;
- motivo o justificacion cuando aplique.

25.5 Edge Cases

- Los procesos automaticos deben registrarse con identidad de proceso y no como usuario anonimo.
- Si una accion falla despues de un cambio parcial, la auditoria debe reflejar el resultado real de la operacion.

25.6 Invariantes

- Ninguna correccion critica del dominio puede ocurrir sin rastro auditable.
- La auditoria debe permitir reconstruir que paso, quien lo hizo y cuando ocurrio.

25.7 Preguntas abiertas

- Confirmar politica de retencion y consulta de eventos de auditoria para uso administrativo.

25.8 Decisiones pendientes

- 'Decision pendiente' 'No bloqueante': definir ventana de retencion de eventos auditables y nivel de acceso a consulta historica.

26 API de resultados

26.1 Descripcion

Este documento define la integracion conceptual con la API externa de resultados. Su objetivo es dejar claro que espera el sistema de un proveedor externo, como se adapta esa informacion al dominio interno y que limites debe respetar la integracion para no contaminar reglas del juego con el modelo del proveedor.

26.2 Alcance

Aplica a la integracion de resultados de partidos y metadatos asociados necesarios para soportar scoring, cierre funcional del partido y trazabilidad. No fija todavia proveedor comercial definitivo ni su contrato tecnico exacto.

26.3 Definiciones

- **Proveedor de resultados**: servicio externo que entrega datos oficiales o candidatos a oficiales sobre partidos.
- **Adaptacion**: transformacion del dato externo al modelo interno del dominio.
- **Resultado vigente**: valor oficial que el sistema reconoce para scoring y vistas derivadas.

26.4 Reglas

- El modelo interno del dominio es la referencia del sistema; la API externa debe adaptarse a ese modelo y no al reves.
- Todo dato recibido desde el proveedor debe poder rastrearse hasta su fuente, momento de recepcion y partido interno asociado.
- La API externa no se considera autoridad absoluta; su salida puede ser validada, ignorada o reemplazada por operacion manual auditada.
- El sistema solo debe consumir del proveedor la informacion necesaria para operar la quiniela y no depender de campos irrelevantes del modelo externo.

26.4.1 Datos minimos esperados del proveedor

- identificador externo del partido;
- fecha y hora oficial del kickoff;
- equipos involucrados;
- estado funcional del partido o equivalente interpretable;
- goles oficiales utilizable para la regla de 90 minutos mas agregado;
- marca temporal de actualizacion del dato externo.

26.4.2 Adaptacion al dominio interno

- El proveedor puede usar nomenclaturas de estado distintas; el backend debe traducirlas a los estados funcionales internos del proyecto.
- El sistema debe identificar un partido interno de forma unica antes de aceptar un resultado externo como candidato valido.
- Si el proveedor incluye informacion no utilizable por el reglamento, como tiempo extra o penales, esa informacion no debe contaminar el resultado oficial usado para scoring.
- La adaptacion debe producir un valor interpretable por el dominio aun cuando el proveedor entregue estructuras mas complejas que las necesarias para la quiniela.

26.4.3 Criterios de aceptacion de un resultado externo

- el partido interno asociado existe;
- el dato puede mapearse al estado funcional del partido;

- los goles y el contexto permiten determinar un resultado compatible con el reglamento;
- no existe override manual vigente que impida adoptar automaticamente el valor de API.

26.4.4 Superficies internas consumidoras

- modulo `results`: adapta y conserva referencia de fuente;
- modulo `matches`: actualiza estado funcional del partido cuando aplique;
- modulo `scoring`: consume solo el resultado vigente ya adaptado;
- frontend: consume el resultado visible oficial y nunca el payload externo crudo.

26.5 Edge Cases

- Si el proveedor entrega un resultado parcial o transitorio, ese dato no debe consolidarse como resultado final sin pasar por la logica interna de aceptacion.
- Si el proveedor cambia el kickoff oficial, el sistema debe tratar ese cambio como dato integrable del partido y no como simple texto informativo.
- Si el proveedor no puede mapear limpiamente el partido o su estado, el dato debe quedar rechazado o pendiente de revision operativa.

26.6 Invariantes

- La API externa no redefine reglas del dominio.
- Todo resultado vigente usado por scoring debe tener fuente identificable.
- El frontend no consume directamente la estructura del proveedor externo.
- Un partido no puede tener simultaneamente mas de un resultado vigente a efectos del sistema.

26.7 Preguntas abiertas

- Confirmar proveedor oficial, frecuencia operativa de consulta y limites de uso del contrato externo definitivo.
- Confirmar si el sistema recibira tambien eventos de fixture y cambios de kickoff desde la misma integracion o desde una fuente separada.

26.8 Decisiones pendientes

- `Decision pendiente` `Bloquea backend`: seleccionar proveedor oficial y documentar su contrato tecnico minimo una vez aprobado.
- `Decision pendiente` `No bloqueante`: definir si el sistema almacenara payload externo completo para auditoria o solo los campos adaptados y referencia de fuente.

27 Sincronizacion y fallback manual

27.1 Descripcion

Este documento define la estrategia funcional de sincronizacion automatica de resultados y el uso de fallback manual cuando la integracion externa no sea suficiente. Su objetivo es dejar clara la prioridad de fuentes, las condiciones de override y el disparo controlado de recalculos.

27.2 Alcance

Aplica a jobs programados, sincronizaciones manuales, overrides administrativos, reintentos y activacion de recalculo a partir de cambios en resultados o estados de partido.

27.3 Definiciones

- **Sincronizacion automatica**: consulta programada o gatillada a la fuente externa de resultados.
- **Fallback manual**: intervencion administrativa para registrar, corregir o preservar un resultado cuando la fuente automatica no es suficiente.
- **Override manual**: resultado vigente establecido por una accion administrativa autorizada.

27.4 Reglas

- Toda actualizacion de resultado debe ser trazable, idempotente y consistente con el dominio.
- Toda correccion manual debe quedar auditada con actor, momento, valor anterior, valor nuevo y justificacion.
- Cuando exista un override manual vigente, la sincronizacion automatica no puede sobrescribirlo sin una accion explicita y auditada.
- Todo cambio efectivo en resultado vigente o estado relevante del partido debe habilitar un recalculo consistente del scoring y leaderboard.

27.4.1 Secuencia funcional de sincronizacion automatica

1. El sistema consulta al proveedor externo segun su ventana operativa definida.
2. Cada dato recibido se adapta al modelo interno del dominio.
3. El sistema determina si el dato es aceptable como candidato para actualizar el resultado vigente.
4. Si el valor no cambia o no es aplicable, la sincronizacion termina sin alterar scoring.
5. Si el valor cambia y no existe override manual vigente, el sistema actualiza el resultado vigente, registra evidencia y habilita recalculo.

27.4.2 Uso del fallback manual

- El fallback manual se usa cuando la fuente externa no entrega el dato a tiempo, lo entrega de forma incompleta, produce inconsistencia o requiere correccion operativa.
- La operacion manual debe exigir autorizacion y justificacion visible.
- El valor manual pasa a ser la verdad oficial vigente hasta nueva accion explicita y auditada.

- El fallback manual no elimina la historia del dato anterior ni la referencia a la fuente de origen.

27.4.3 Eventos que deben habilitar recalculo

- publicacion inicial de resultado oficial utilizable;
- correccion de resultado vigente;
- cambio de estado de partido que altere scoring o cierre funcional;
- override manual que reemplace el valor previamente vigente.

27.4.4 Politica de prioridad de fuentes

1. Si no existe override manual vigente, el sistema puede adoptar el valor aceptado del proveedor externo.
2. Si existe override manual vigente, ese valor tiene prioridad operativa sobre la API automatica.
3. Para volver a aceptar automaticamente un valor externo luego de un override manual, debe existir una nueva accion explicita y auditada.

27.4.5 Ventanas funcionales de sincronizacion

La estrategia exacta de frecuencia queda pendiente de proveedor e infraestructura, pero funcionalmente el sistema debe cubrir:

- sincronizacion previa a kickoff para cambios oficiales de horario o estado;
- sincronizacion cercana al cierre del partido para detectar resultado utilizable;
- re-sincronizacion posterior para detectar correcciones;
- sincronizacion manual administrativa cuando operacion lo requiera.

27.5 Edge Cases

- Si un partido fue postergado antes del lock, la sincronizacion del nuevo kickoff debe reflejarse sin perder consistencia con picks.
- Si un partido fue suspendido, la sincronizacion no debe cerrar scoring hasta existir resultado oficial utilizable.
- Si una correccion manual reemplaza un dato de API y luego la API vuelve a sincronizar, el valor manual debe preservarse hasta nueva accion explicita.
- Si el mismo evento provoca mas de un recalculo, el estado final debe converger al mismo resultado por idempotencia.

27.6 Invariantes

- El fallback manual no elimina trazabilidad.
- La integracion automatica no puede sobrescribir un override manual vigente por su sola ejecucion.

- Todo recalculo parte del resultado vigente oficial y no de valores intermedios de sincronizacion.
- El sistema no debe dejar dos verdades simultaneas activas para un mismo partido.

27.7 Preguntas abiertas

- Confirmar ventanas reales de scheduler, reintentos y re-sincronizacion una vez elegido el proveedor definitivo.
- Confirmar si habra una accion administrativa explicita para liberar un override manual y volver a aceptar sincronizacion automatica normal.

27.8 Decisiones pendientes

- `Decision pendiente No bloqueante`: definir frecuencia exacta de sincronizacion segun limites del proveedor y necesidades operativas.
- `Decision pendiente No bloqueante`: decidir si el recalculo posterior a sincronizacion se ejecuta inmediatamente en todos los casos o si algunas rutas usaran cola diferida.

28 Email y notificaciones

28.1 Descripcion

Este documento define las comunicaciones funcionales minimas de v1 producidas por el sistema. Su objetivo es identificar que eventos ameritan correo o notificacion visible, con que intencion y bajo que limites, evitando una estrategia de mensajeria excesiva o ambigua.

28.2 Alcance

Aplica a correos transaccionales y notificaciones funcionales del producto. No define copy final detallado ni un sistema avanzado de preferencias de comunicacion.

28.3 Definiciones

- **Notificacion accionable**: mensaje que impulsa una accion concreta del usuario o del admin.
- **Correo transaccional**: email disparado por un cambio de estado o una accion funcional del sistema.
- **Notificacion visible**: mensaje mostrado dentro del producto para explicar un estado, cambio o restriccion relevante.

28.4 Reglas

- Las comunicaciones de v1 deben ser pocas, claras y estrictamente funcionales.
- Ninguna notificacion debe prometer un estado que el sistema no haya persistido y confirmado.
- Los mensajes deben respetar los estados reales del usuario, partido o resultado.
- Las acciones administrativas internas no deben generar ruido innecesario al participante salvo cuando cambien su experiencia funcional.

28.4.1 Correos minimos esperados en v1

Evento	Destinatario	Objetivo funcional
registro de cuenta	participante	confirmar alta e indicar siguientes pasos de acceso o activacion
recuperacion de contrasena	participante	permitir continuar el flujo de recuperacion
comprobante recibido	participante	confirmar recepcion para revision administrativa
activacion aprobada	participante	informar que la cuenta ya puede participar
activacion rechazada	participante	informar resolucion y siguiente accion esperada si aplica
bloqueo administrativo de cuenta	participante	informar restriccion funcional de acceso

28.4.2 Notificaciones visibles dentro del producto

- confirmacion o rechazo de guardado de pick;
- restriccion por lock vencido;
- restriccion por cuenta no activa;
- confirmacion o rechazo de guardado de prediccion especial;
- aviso explicito de resultado corregido cuando esa superficie sea definida;
- feedback de accion administrativa dentro del panel admin.

28.4.3 Reglas de disparo

- Un correo de activacion aprobada solo se envia cuando el estado del usuario ya fue persistido como `ACTIVO`.
- Un correo de rechazo solo se envia cuando la resolucion administrativa ya fue aplicada al usuario.
- Las notificaciones visibles de lock o permiso deben poder dispararse tanto por validacion previa de interfaz como por rechazo recibido del backend.
- Un aviso por resultado corregido debe dispararse solo cuando el sistema ya consolidó el nuevo resultado vigente y habilito el recalculo correspondiente.

28.4.4 Mensajes que no son prioridad para v1

- recordatorios promocionales;
- notificaciones masivas por cada partido proximo a comenzar;
- resúmenes automaticos complejos de rendimiento;

- preferencias avanzadas de suscripcion por canal.

28.5 Edge Cases

- Si un comprobante se vuelve a cargar o corregir, el sistema debe evitar enviar confirmaciones redundantes sin valor funcional.
- Si una accion falla tecnicamente despues de que el usuario inicia el flujo, no debe enviarse un correo de exito.
- Si un resultado se corrige mas de una vez, la politica de aviso visible debe evitar ruido excesivo aunque preserve transparencia.

28.6 Invariantes

- Ninguna comunicacion contradice el estado persistido del sistema.
- Ningun correo de exito se emite antes de confirmar la transicion funcional correspondiente.
- La mensajeria de v1 se mantiene funcional y acotada, no promocional.

28.7 Preguntas abiertas

- Confirmar si v1 incluira correo explicito para resultado corregido o si ese evento vivira solo como notificacion visible dentro del producto.
- Confirmar si el panel admin requiere notificaciones internas persistentes ademas del feedback inmediato de interfaz.

28.8 Decisiones pendientes

- `Decision pendiente` `No bloqueante`: cerrar la lista final de correos obligatorios y opcionales de v1 con su dueño funcional.
- `Decision pendiente` `Bloquea frontend`: definir en que superficies y durante cuanto tiempo se muestra el aviso de resultado corregido al participante.

29 Roles y permisos

29.1 Descripcion

Este documento define la matriz funcional de roles y permisos del sistema. Su objetivo es aplicar *Least Privilege* al producto y dejar claro que acciones puede ejecutar cada tipo de usuario sin mezclar permisos funcionales con estados de cuenta.

29.2 Alcance

Aplica a participantes y operadores administrativos del sistema. No define el mecanismo tecnico final de autorizacion, pero si la frontera funcional de acceso.

29.3 Definiciones

- **Rol**: conjunto de permisos funcionales asignados a un tipo de actor.
- **Permiso**: accion autorizada sobre una capacidad del sistema.
- **Least Privilege**: principio por el cual cada actor recibe solo el nivel minimo de acceso necesario.

29.4 Reglas

- El sistema debe separar claramente permisos de rol y estado funcional del usuario.
- Un participante `ACTIVO` no adquiere privilegios administrativos por su estado.
- Toda accion administrativa sensible debe exigir autorizacion valida y dejar auditoria.
- Los permisos deben definirse por capacidad funcional, no por conveniencia de interfaz.

29.4.1 Roles funcionales minimos

- `Participante`: usuario final de la quiniela.
- `Admin`: operador autorizado para gestion del torneo y resolucion operativa.
- `Super admin`: rol administrativo con mayor amplitud para configuracion critica o acciones de alto impacto.

29.4.2 Matriz funcional de permisos

Capacidad	Participante	Admin	Super admin
registrarse e iniciar sesion	si	no aplica	no aplica
consultar estado de cuenta propio	si	si	si
cargar comprobante propio	si	no	no
capturar picks propios en ventana abierta	si	no	no
capturar predicciones especiales propias en ventana abierta	si	no	no
consultar leaderboard y resultados visibles	si	si	si
aprobar o rechazar comprobantes	no	si	si
bloquear usuarios	no	si	si
actualizar kickoff o estado de partido	no	si	si
cargar o corregir resultado manualmente	no	si	si
ejecutar recalculo	no	si	si
consultar auditoria operativa	no	si	si
archivar torneo	no	no	si

gestionar configuracion del sistema	critica	no	no	si	

29.4.3 Reglas de interpretacion

- El rol de participante se combina con estados funcionales como `PENDIENTE`, `ACTIVO`, `RECHAZADO` o `BLOQUEADO`.
- Un participante no `ACTIVO` puede autenticarse y consultar su estado cuando corresponda, pero no ejecutar acciones de juego reservadas a cuentas activas.
- El rol administrativo no debe usarse para jugar con privilegios especiales ni para eludir restricciones del dominio.
- El archivado de torneo y configuraciones criticas quedan reservados al nivel mas alto de autorizacion.

29.5 Edge Cases

- Un admin puede tener cuenta propia en el sistema, pero su experiencia de participante y su experiencia administrativa deben permanecer separadas funcionalmente.
- Si una accion administrativa afecta datos competitivos, como resultado, scoring o estado de usuario, debe registrarse con justificacion cuando aplique.
- Si el frontend expone un control visible por error, el backend debe seguir rechazando la operacion sin permiso.

29.6 Invariantes

- Ningun rol recibe permisos mas amplios de lo necesario para su operacion.
- Ninguna accion administrativa critica existe sin posibilidad de auditoria.
- El permiso de ejecutar una accion no sustituye validaciones de estado del dominio.

29.7 Preguntas abiertas

- Confirmar la diferencia exacta de capacidades entre `admin` y `super admin` si el proyecto decide reducir a un solo rol administrativo en v1.

29.8 Decisiones pendientes

- `Decision pendiente No bloqueante`: confirmar si v1 requiere realmente `super admin` como rol separado o si basta un unico rol administrativo con politicas operativas adicionales.
- `Decision pendiente No bloqueante`: definir si ciertas acciones administrativas, como recalculo global o archivado, requeriran doble confirmacion funcional ademas de autorizacion.

30 Seguridad aplicativa

30.1 Descripcion

Este documento define los lineamientos de seguridad aplicativa del proyecto alineados con *Secure by Design*, buenas practicas OWASP y principios de produccion ya aprobados para Quiniela.

30.2 Alcance

Aplica a autenticacion, autorizacion, validacion de entradas, proteccion de configuracion, hardening base y manejo seguro de errores. No sustituye una auditoria de seguridad ni una politica de infraestructura detallada.

30.3 Definiciones

- **Secure by Design**: seguridad incorporada desde el diseno y no agregada al final.
- **Superficie de ataque**: puntos donde el sistema recibe datos, ejecuta acciones o expone informacion.
- **Hardening**: medidas de reduccion de riesgo en configuracion y comportamiento del sistema.

30.4 Reglas

- Todo input externo debe validarse de forma estricta en la frontera de entrada.
- Toda accion critica debe exigir autenticacion valida, autorizacion acorde al rol y trazabilidad suficiente.
- La configuracion sensible debe vivir fuera del codigo y gestionarse por entorno.
- Los errores expuestos al cliente no deben filtrar detalles internos innecesarios.
- La seguridad no puede debilitar reglas del dominio; debe reforzarlas.

30.4.1 Controles minimos esperados

- validacion estricta de payloads y parametros;
- manejo seguro de sesion o tokens;
- proteccion de rutas administrativas;
- headers y hardening HTTP apropiados para produccion;
- politicas de contrasena y recuperacion de acceso controladas;
- proteccion contra abuso basico en autenticacion y acciones sensibles;
- sanitizacion y manejo seguro de archivos o evidencias cargadas si aplica.

30.4.2 Autenticacion y autorizacion

- La autenticacion debe identificar de forma fiable al actor antes de permitir operaciones protegidas.
- La autorizacion debe evaluarse por rol y por estado funcional cuando la accion lo requiera.

- Una cuenta autenticada pero no `ACTIVA` no puede ejecutar acciones competitivas reservadas a jugadores habilitados.
- Las operaciones administrativas no deben confiar en ocultamiento de UI; su proteccion principal vive en backend.

30.4.3 Validacion y manejo de entrada

- Los contratos de entrada deben definirse de forma explicita y rechazarse si no cumplen formato, rango o contexto funcional.
- El rango de goles permitido para picks debe respetar la regla oficial del producto: enteros entre `0` y `9`.
- Una entrada formalmente valida puede seguir siendo rechazada por estado del dominio, por ejemplo lock vencido o cuenta no activa.
- Las validaciones del frontend son complementarias; el backend es la autoridad final.

30.4.4 Configuracion y secretos

- Ningun secreto debe quedar hardcodeado en codigo fuente ni en documentos de ejemplo.
- La configuracion operativa debe separarse por entorno y mantenerse fuera del repositorio cuando sea sensible.
- Los valores por defecto del sistema no deben abrir superficies inseguras en produccion.

30.4.5 Manejo de errores

- Los errores funcionales deben informar rechazo de forma comprensible sin exponer detalle tecnico sensible.
- Los errores tecnicos deben registrarse con suficiente contexto interno para diagnostico.
- Una respuesta de error no debe filtrar internals de base de datos, stack trace ni configuracion.

30.5 Edge Cases

- Si el frontend intenta una accion permitida por interfaz pero rechazada por backend, el sistema debe preferir seguridad y reconciliar la vista.
- Si una carga de comprobante o archivo existe en v1, su validacion y almacenamiento deben considerarse superficie sensible.
- Si una accion administrativa modifica resultados o scoring, la proteccion debe cubrir tanto acceso como auditoria posterior.

30.6 Invariantes

- Ninguna accion critica opera sin autenticacion, autorizacion y trazabilidad acordes.
- Ningun input externo cruza la frontera del sistema sin validacion.
- Ninguna respuesta publica expone internals innecesarios del sistema.

- La configuracion sensible permanece fuera del codigo fuente.

30.7 Preguntas abiertas

- Confirmar el checklist final de controles obligatorios de v1, incluyendo politica de rate limiting, manejo de archivos y nivel exacto de protecciones HTTP.

30.8 Decisiones pendientes

- `Decision pendiente` `Bloquea backend`: cerrar el set minimo obligatorio de controles de autenticacion, gestion de sesion y proteccion contra abuso para v1.
- `Decision pendiente` `No bloqueante`: definir si el proyecto documentara una checklist OWASP interna separada o si estos controles vivirán solo en este documento.

31 Logging, observabilidad y auditoria

31.1 Descripcion

Este documento define la relacion entre logs estructurados, observabilidad tecnica y auditoria funcional. Su objetivo es dejar claro que problemas resuelve cada uno, donde se complementan y como evitar duplicacion o lagunas de trazabilidad.

31.2 Alcance

Aplica a eventos tecnicos, correlacion operativa, rastreo de errores y acciones funcionales criticas del negocio. No define dashboards concretos ni una implementacion detallada de telemetria.

31.3 Definiciones

- **Log estructurado**: evento tecnico emitido en formato consistente y parseable.
- **Observabilidad**: capacidad de entender el comportamiento del sistema a partir de señales tecnicas.
- **Auditoria**: registro funcional de acciones relevantes para negocio, operacion y trazabilidad.
- **Correlacion**: capacidad de relacionar multiples eventos tecnicos y funcionales dentro de una misma operacion.

31.4 Reglas

- Logs, observabilidad y auditoria son complementarios y no deben confundirse.
- Un log tecnico no sustituye un evento de auditoria cuando existe impacto funcional relevante.
- Toda accion critica del sistema debe dejar al menos el rastro tecnico y funcional necesario para ser explicada.
- Los logs deben emitirse como flujo de eventos y no como archivos locales acoplados al proceso, en linea con Twelve-Factor App.

31.4.1 Responsabilidad de cada capa

- Logs estructurados: diagnostico tecnico, errores, latencia, resultados de integracion y ejecucion de procesos.
- Observabilidad: metricas, trazas y correlacion para entender salud y comportamiento del sistema.
- Auditoria: evidencia funcional de acciones criticas como aprobacion de pagos, bloqueos, overrides, recalculos y archivado.

31.4.2 Eventos funcionales que deben auditarse

- aprobacion o rechazo de comprobantes;
- bloqueo de usuarios;
- actualizacion manual de kickoff o estado de partido;
- carga o correccion manual de resultados;
- ejecucion de recalculo;
- archivado del torneo;
- cualquier accion administrativa que altere scoring, acceso o verdad oficial vigente.

31.4.3 Campos minimos esperados

Para logs tecnicos

- timestamp;
- nivel;
- modulo o servicio;
- tipo de evento;
- identificador de correlacion cuando exista;
- contexto tecnico suficiente para diagnostico sin filtrar secretos.

Para eventos de auditoria

- timestamp;
- actor;
- accion;
- entidad afectada;
- valor anterior y nuevo cuando aplique;
- justificacion o motivo cuando corresponda.

31.4.4 Correlacion esperada

- Una operacion critica debe poder relacionar request, proceso interno, integracion externa y efecto funcional cuando aplique.
- Un error tecnico que cause impacto funcional debe ser rastreable hacia el proceso o evento funcional afectado.
- La auditoria debe permitir explicar por que un valor oficial cambio, mientras que la observabilidad debe ayudar a explicar como ocurrio tecnicamente.

31.5 Edge Cases

- Una correccion de resultado puede requerir log tecnico, evento de auditoria y telemetria de recalculo sin que esos tres mecanismos repitan exactamente el mismo payload.
- Un error de integracion externa puede requerir alerta tecnica sin generar por si solo un evento de auditoria, salvo que derive en una accion manual relevante.
- Si una operacion administrativa falla antes de persistir cambios, puede requerir log tecnico y evidencia operativa sin convertirse en evento funcional exitoso.

31.6 Invariantes

- Ningun evento critico queda sin rastro tecnico o funcional suficiente.
- La auditoria funcional no se usa como reemplazo de logs tecnicos.
- Los mecanismos de trazabilidad no deben exponer secretos ni datos innecesarios en texto plano.

31.7 Preguntas abiertas

- Confirmar la estrategia final de correlacion entre logs, trazas y audit events, incluyendo si existira un identificador comun de operacion.

31.8 Decisiones pendientes

- `Decision pendiente No bloqueante`: definir el catalogo final de eventos auditables y su esquema exacto una vez cierre la operacion administrativa real de v1.
- `Decision pendiente No bloqueante`: confirmar el nivel minimo de telemetria y alertas que se documentara formalmente en una iteracion posterior.

32 Contingencia de resultados

32.1 Descripcion

Este documento define el plan operativo minimo cuando falle la integracion automatica de resultados o cuando los datos externos no sean suficientes para sostener la operacion del torneo con consistencia y trazabilidad.

32.2 Alcance

Aplica a indisponibilidad del proveedor, datos inconsistentes, ausencia de actualizacion o necesidad de correccion manual en resultados y estados de partido.

32.3 Definiciones

- **Contingencia**: procedimiento excepcional para sostener la operacion del negocio sin depender del flujo ordinario automatizado.
- **Evento de contingencia**: situacion que obliga a evaluar o ejecutar fallback manual.
- **Resolucion operativa**: accion administrativa explicita para preservar una verdad oficial vigente del sistema.

32.4 Reglas

- El sistema debe poder operar con fallback manual cuando la API externa falle o no entregue un valor utilizable a tiempo.
- Toda contingencia debe quedar documentada y auditada.
- Una contingencia no autoriza a relajar reglas de dominio, scoring ni trazabilidad.
- Si se establece un resultado manual durante contingencia, ese valor sigue la misma regla de prioridad operativa de un override manual autorizado.

32.4.1 Eventos que pueden activar contingencia

- indisponibilidad total del proveedor externo;
- respuesta incompleta o inconsistente respecto del partido esperado;
- ausencia de actualizacion dentro de la ventana operativa razonable;
- correccion urgente requerida para preservar consistencia competitiva;
- conflicto entre dato externo y evidencia operativa validada por administracion.

32.4.2 Secuencia minima de contingencia

1. Detectar y registrar el evento de contingencia.
2. Verificar si el problema puede resolverse con reintento o re-sincronizacion controlada.
3. Si no basta la integracion automatica, escalar a operacion administrativa autorizada.
4. Registrar resultado o estado manual con justificacion y evidencia suficiente.
5. Ejecutar o habilitar recalculo si el valor vigente cambió.
6. Mantener trazabilidad del evento hasta su cierre operativo.

32.4.3 Responsabilidad operativa minima

- La deteccion puede venir de monitoreo tecnico o revision administrativa.

- La resolucion manual debe ejecutarla un actor con permisos administrativos suficientes.
- Toda accion de contingencia debe dejar contexto claro para posteriores revisiones o disputas.

32.4.4 Comunicacion funcional

- La contingencia interna no exige comunicar todos los detalles operativos al participante.
- Si la contingencia produce correccion del resultado visible, el usuario debe recibir el aviso funcional ya definido para resultado corregido.
- La comunicacion hacia participantes debe priorizar transparencia funcional sin exponer ruido operativo innecesario.

32.5 Edge Cases

- Si la API vuelve a estar disponible despues de una carga manual, el valor manual no debe perderse por sincronizacion automatica.
- Si el partido queda suspendido sin resultado utilizable, la contingencia puede documentar el estado pero no forzar scoring inexistente.
- Si ocurre mas de una contingencia sobre el mismo partido, la auditoria debe permitir reconstruir el orden de decisiones y el valor vigente final.

32.6 Invariantes

- La contingencia no puede sacrificar trazabilidad ni consistencia del scoring.
- Ninguna contingencia crea dos resultados vigentes simultaneos.
- Todo cambio manual ejecutado en contingencia requiere actor identificado y justificacion.

32.7 Preguntas abiertas

- Confirmar si la operacion real de v1 definira una ventana temporal objetiva para declarar formalmente una contingencia de resultados.

32.8 Decisiones pendientes

- `Decision pendiente No bloqueante`: definir responsables nominales y tiempos objetivo de respuesta para contingencias reales de v1.
- `Decision pendiente No bloqueante`: decidir si existira una plantilla operativa separada para registro de contingencias fuera de la auditoria del sistema.

33 Estrategia de testing

33.1 Descripcion

Este documento define la estrategia general de pruebas del proyecto Quiniela. Su objetivo es asegurar que las reglas del dominio, los procesos criticos y los flujos visibles del producto se validen con el nivel de prueba adecuado segun su riesgo funcional y operativo.

33.2 Alcance

Aplica a pruebas unitarias, de integracion, end to end y validacion operativa. Cubre backend, frontend e integraciones en el nivel necesario para proteger reglas del negocio y confianza competitiva del sistema.

33.3 Definiciones

- **Prueba unitaria**: validacion acotada de una regla o componente aislado.
- **Prueba de integracion**: validacion del comportamiento conjunto entre modulos, capas o contratos.
- **Prueba end to end**: validacion de un flujo funcional completo desde la perspectiva del usuario o del operador.
- **Prueba operativa**: validacion de procesos o contingencias relevantes para salida a operacion.

33.4 Reglas

- El testing debe priorizar riesgo funcional antes que volumen de casos triviales.
- Las reglas que puedan producir disputa competitiva o error operativo deben tener cobertura prioritaria.
- Las pruebas deben distribuirse por nivel segun donde viva realmente el riesgo: dominio, contrato, flujo o integracion.
- Una validacion del frontend no reemplaza una prueba del enforcement real del backend.

33.4.1 Prioridades de riesgo

- Prioridad alta: activacion de usuarios, lock de picks, scoring, resultados corregidos, recalculo, permisos administrativos y archivado.
- Prioridad media: visualizacion de leaderboard, feedback funcional, flujos de perfil y recuperacion de acceso.
- Prioridad baja: detalles de presentacion no normativos, siempre que no alteren acciones criticas.

33.4.2 Distribucion recomendada por nivel

Nivel	Enfoque	Que debe proteger
unitario	reglas acotadas y transformaciones puras	scoring, desempates, validaciones de rango, traduccion de estados, adaptacion de resultados
integracion	colaboracion entre modulos y contratos	activacion por comprobante, locks, sincronizacion de resultados, recalculo, permisos y respuestas de API
end to end	flujo visible completo	registro, activacion, captura de picks, bloqueo, leaderboard, admin y correccion manual

operativo	continuidad y contingencia	fallback manual, override, archivado y trazabilidad minima para operacion

33.4.3 Cobertura esperada por area

- Backend: enfasis en reglas del dominio, procesos criticos, validacion de frontera, idempotencia y contratos conceptuales.
- Frontend: enfasis en flujos visibles, estados de pantalla, validacion de experiencia y restricciones funcionales recibidas desde backend.
- Integraciones: enfasis en adaptacion de proveedor externo, prioridad de override manual y tolerancia a datos incompletos o tardios.

33.4.4 Herramientas base aprobadas

- `Vitest` para pruebas unitarias y parte de integracion backend.
- `Playwright` para pruebas end to end del frontend y flujos principales.
- Validacion de contratos con `Zod` como apoyo al enforcement, sin sustituir pruebas funcionales.

33.5 Edge Cases

- Un flujo puede requerir mas de un nivel de prueba para quedar realmente protegido; por ejemplo lock de picks debe existir en backend y en end to end.
- Un caso simple de UI puede volverse critico si comunica incorrectamente un estado de activacion, resultado corregido o restriccion por lock.
- Una integracion estable puede seguir necesitando pruebas de contingencia por su impacto operativo.

33.6 Invariantes

- Ningun flujo critico se considera listo sin criterio de prueba verificable.
- Ninguna regla clave del dominio depende de una sola prueba de interfaz.
- Las pruebas deben poder explicar por que una regla critica esta protegida y en que nivel lo esta.

33.7 Preguntas abiertas

- Confirmar el umbral minimo de cobertura formal que el proyecto exigira por repositorio y por nivel de prueba.

33.8 Decisiones pendientes

- `Decision pendiente` `No bloqueante`: definir metas cuantitativas de cobertura por backend y frontend si el proyecto decide imponerlas.
- `Decision pendiente` `No bloqueante`: confirmar si las pruebas operativas de contingencia viviran dentro de suites automatizadas o como checklist manual controlado.

34 Casos criticos de prueba

34.1 Descripcion

Este documento define el inventario minimo de escenarios de negocio que no deben romperse. Su objetivo es proteger la justicia competitiva, la consistencia operativa y la confianza del sistema sobre los puntos de mayor riesgo del producto.

34.2 Alcance

Aplica a activacion, picks, lock, resultados, scoring, leaderboard, administracion y contingencias relevantes para v1.

34.3 Definiciones

- **Caso critico**: escenario cuya falla afecta justicia, operacion o confianza del sistema.
- **Resultado esperado**: comportamiento verificable que debe cumplirse sin ambiguedad.

34.4 Reglas

- Los casos criticos se definen por impacto funcional, no solo por complejidad tecnica.
- Toda regla critica del juego debe tener al menos un caso de prueba asociado.
- Los casos deben poder mapearse claramente al documento responsable de la regla que protegen.

34.4.1 Inventario minimo de casos criticos

ID	Escenario	Area principal	Resultado esperado
CT-01	usuario `PENDIENTE` intenta guardar pick	activacion y picks	operacion rechazada por estado funcional
CT-02	usuario `ACTIVO` guarda pick valido antes del lock	picks	pick persistido correctamente
CT-03	usuario intenta editar pick despues del lock	locks	operacion rechazada y pick permanece bloqueado
CT-04	partido llega al lock sin prediccion guardada	locks y scoring	se interpreta `SIN_PICK` y no se fabrica valor
CT-05	prediccion especial intenta editarse despues del cierre del primer partido	predicciones especiales	operacion rechazada por lock especial
CT-06	resultado oficial llega y se calcula scoring base	scoring	puntaje por outcome y exacto coincide con la regla
CT-07	partido de fase con multiplicador aplica precision exacta	scoring	total calculado sin redondeo interno arbitrario

CT-08	partido cancelado sin resultado utilizable	Edge Cases	no genera puntaje para ningun usuario
CT-09	resultado corregido dispara recalculo	resultados y leaderboard	ranking y puntajes se actualizan desde verdad vigente
CT-10	override manual vigente recibe nueva sincronizacion automatica	integraciones	el valor manual no es sobrescrito automaticamente
CT-11	admin aprueba comprobante valido	activacion admin	usuario pasa a `ACTIVO` con auditoria
CT-12	admin rechaza comprobante	activacion admin	usuario pasa a `RECHAZADO` con trazabilidad
CT-13	accion administrativa sin permisos suficientes	seguridad	operacion rechazada y sin cambio de estado
CT-14	archivado del torneo bloquea operacion competitiva ordinaria	admin y operacion	competencia queda en modo solo lectura

34.4.2 Casos criticos adicionales recomendados

- cambio de kickoff antes del lock recalcula correctamente la ventana abierta;
- postergacion despues del lock preserva picks ya bloqueados salvo accion extraordinaria auditada;
- leaderboard muestra total acumulado coherente con desglose cuando ese desglose se exponga;
- mensaje funcional por resultado corregido aparece en la superficie definida sin exponer detalle operativo excesivo.

34.4.3 Asignacion por nivel de prueba

- `CT-01` a `CT-07`: backend e integracion como minimo, con apoyo end to end en flujos principales.
- `CT-08` a `CT-10`: integracion y pruebas operativas por su dependencia de estados no ordinarios.
- `CT-11` a `CT-14`: integracion y end to end administrativo.

34.5 Edge Cases

- Un mismo caso critico puede requerir mas de una prueba si existe riesgo tanto en backend como en frontend.
- Algunos casos criticos dependen de datos temporales o scheduler; en esos casos la automatizacion puede requerir dobles o fixtures controlados.
- Un caso visible en UI no se considera cubierto si falta verificacion del comportamiento real del backend.

34.6 Invariantes

- Ninguna regla critica del juego queda sin un caso de prueba asociado.

- Todo caso critico debe tener resultado esperado verificable.
- Los escenarios de lock, scoring y correccion de resultados mantienen prioridad alta en cualquier iteracion.

34.7 Preguntas abiertas

- Confirmar si el proyecto quiere formalizar una matriz trazable completa entre cada caso critico y su tipo de prueba exacto por repositorio.

34.8 Decisiones pendientes

- `Decision pendiente` `No bloqueante`: decidir si se agregara una codificacion oficial de casos de prueba para rastreo transversal entre repositorios.
- `Decision pendiente` `No bloqueante`: definir si v1 exigira cobertura automatizada obligatoria para todos los casos de este inventario o si algunos podran validarse manualmente en salida controlada.

35 Criterios de aceptacion

35.1 Descripcion

Este documento define los criterios de aceptacion funcional y la base de UAT para Quiniela. Su objetivo es dejar claro cuando un flujo puede considerarse aceptable para trabajo y salida controlada, evitando criterios vagos o dependientes de interpretacion libre.

35.2 Alcance

Aplica a validacion funcional de participante, administracion y operacion minima de v1. No reemplaza la estrategia de testing tecnica, sino que define la barra de aceptacion observable del producto.

35.3 Definiciones

- **Criterio de aceptacion**: condicion observable y verificable que un flujo debe cumplir.
- **UAT**: validacion con responsables del proyecto o usuarios representativos antes de considerar un flujo aceptado.

35.4 Reglas

- Todo criterio de aceptacion debe ser observable, verificable y trazable a una regla del producto.
- Ningun criterio debe depender de interpretacion subjetiva abierta.
- Los criterios de aceptacion deben cubrir comportamiento normal, restricciones funcionales y resultado esperado ante rechazo.

35.4.1 Criterios minimos por flujo

Registro y acceso

- Un usuario puede registrarse con datos validos y queda en estado `PENDIENTE`.
- Un usuario puede iniciar sesion con credenciales validas.
- Un usuario puede iniciar el flujo de recuperacion de acceso.

Activacion

- Un usuario `PENDIENTE` puede registrar comprobante.
- La resolucion administrativa aprobada cambia al usuario a `ACTIVO`.
- La resolucion administrativa rechazada cambia al usuario a `RECHAZADO`.
- Un usuario no `ACTIVO` no puede participar en flujos de picks.

Picks y predicciones especiales

- Un usuario `ACTIVO` puede crear y editar picks antes del lock.
- Un usuario no puede editar picks despues del lock del partido.
- Un partido sin pick guardado al llegar el lock se interpreta como `SIN_PICK`.
- Las predicciones especiales pueden editarse hasta cinco minutos antes del kickoff del primer partido del torneo y luego quedan bloqueadas.

Resultados, scoring y leaderboard

- El sistema calcula correctamente puntos por outcome y marcador exacto.
- El sistema aplica multiplicadores de fase con precision exacta.
- El leaderboard se ordena por puntaje total acumulado.
- Una correccion de resultado actualiza puntajes y ranking conforme al resultado vigente.

Administracion y operacion

- Un admin autorizado puede aprobar o rechazar comprobantes.
- Un admin autorizado puede registrar o corregir resultado manual con trazabilidad.
- Una accion administrativa sin permisos suficientes es rechazada.
- El archivado del torneo deja la competencia en modo solo lectura.

35.4.2 Criterios de UAT minimo

- El responsable funcional puede seguir el flujo de activacion sin ambiguedad.
- El participante puede entender cuando un pick esta editable, bloqueado o ausente.
- El ranking visible coincide con los resultados y reglas definidas.

- El panel admin permite resolver comprobantes y resultados sin comportamiento ambiguo.

35.5 Edge Cases

- Un flujo puede cumplir happy path y seguir sin aceptarse si no comunica bien restricciones funcionales como lock o cuenta no activa.
- Un flujo administrativo puede funcionar tecnicamente pero no aceptarse si no deja trazabilidad suficiente.
- Un criterio puede depender de mas de un repositorio; en esos casos la aceptacion final requiere validacion integrada.

35.6 Invariantes

- No se aceptan criterios ambiguos ni dependientes de interpretacion libre.
- Ningun flujo critico se considera aceptado sin verificar restricciones y rechazos relevantes.
- La aceptacion funcional no contradice reglas ya cerradas del dominio.

35.7 Preguntas abiertas

- Confirmar quien aprueba formalmente cada flujo en UAT y si existira una matriz de firmas o simplemente validacion conjunta de trabajo.

35.8 Decisiones pendientes

- `Decision pendiente` `No bloqueante`: definir la matriz final de aprobacion por flujo y responsable una vez se acerque la salida operativa.
- `Decision pendiente` `No bloqueante`: decidir si el proyecto mantendra una checklist UAT separada y ejecutable fuera de este documento.

36 Principios UI UX

36.1 Descripcion

Este documento define los principios funcionales de interfaz y experiencia para la v1 de Quiniela. Su objetivo es asegurar que la experiencia del producto favorezca claridad, baja friccion y confianza operativa, sin contradecir nunca las reglas reales del dominio.

36.2 Alcance

Aplica a la experiencia de participante y administracion. No define estilo visual final ni identidad grafica completa; define criterios funcionales para construir pantallas correctas y usables.

36.3 Definiciones

- **Accion principal**: CTA dominante de una pantalla.
- **Friccion util**: friccion intencional que protege una accion critica o reduce error operativo.

- **Estado visible**: manifestacion clara de una condicion funcional relevante para el usuario.

36.4 Reglas

- Cada pantalla debe tener una accion principal clara.
- La experiencia debe priorizar comprension del estado actual por encima de densidad de informacion.
- Las restricciones del dominio deben comunicarse de forma visible antes de convertirse en sorpresa para el usuario.
- La interfaz no debe insinuar que una accion es posible si el sistema probablemente la rechazara por lock, permisos o estado funcional.

36.4.1 Principios de experiencia para participante

- El usuario debe entender rapidamente si su cuenta puede jugar o si aun depende de activacion.
- El flujo de picks debe reducir error de captura y dejar claro si un partido esta editable, bloqueado o sin pick.
- El leaderboard debe ser facil de interpretar y no depender de calculos mentales del usuario.
- Las predicciones especiales deben sentirse relacionadas con picks, pero claramente separadas de los picks por partido.

36.4.2 Principios de experiencia para administracion

- Las acciones administrativas criticas deben priorizar seguridad, contexto y trazabilidad por encima de velocidad ciega.
- La interfaz administrativa debe separar claramente resolucion de comprobantes, gestion de resultados y acciones de recalculo.
- Toda accion de alto impacto debe mostrar suficiente contexto para evitar errores operativos.

36.4.3 Patrones funcionales esperados

- CTA unico o dominante por pantalla.
- restricciones visibles antes del intento fallido cuando el frontend ya conoce la limitacion;
- modo solo lectura cuando una entidad ya no es editable;
- feedback inmediato y comprensible tras mutaciones criticas;
- separacion visual entre experiencia de participante y experiencia administrativa.

36.4.4 Friccion aceptable

- confirmacion de acciones administrativas criticas;
- advertencias cuando una accion cambia resultados, scoring o estado de usuarios;
- mensajes claros antes de abandonar una captura relevante si existe riesgo de perdida visible.

36.5 Edge Cases

- Una pantalla puede ser visualmente simple pero funcionalmente incorrecta si esconde locks, restricciones o estados de cuenta.
- Una accion administrativa demasiado rapida puede elevar riesgo operativo aunque reduzca clics.
- Una experiencia de baja friccion no es aceptable si compromete auditabilidad o claridad competitiva.

36.6 Invariantes

- Ninguna decision visual contradice una restriccion funcional del dominio.
- Ninguna pantalla critica carece de accion principal identificable.
- La UX prioriza comprension del estado real del sistema sobre adornos o atajos ambiguos.

36.7 Preguntas abiertas

- Confirmar si la navegacion final de picks y predicciones especiales se resolvera como tabs, rutas separadas o un contenedor unico con secciones diferenciadas.

36.8 Decisiones pendientes

- `Decision pendiente No bloqueante`: definir el patron final de navegacion del modulo de picks sin romper la separacion funcional ya aprobada.
- `Decision pendiente No bloqueante`: confirmar si la experiencia admin de v1 requiere vistas independientes por area o un panel unico con subsecciones.

37 Pantallas y estados

37.1 Descripcion

Este documento define el inventario funcional de pantallas y los estados visibles minimos que cada una debe contemplar. Su objetivo es reducir ambiguedad entre flujo, contrato y comportamiento visible del frontend.

37.2 Alcance

Aplica a vistas publicas, autenticadas y administrativas de v1. No define wireframes finales, pero si que estados deben existir para que la experiencia sea correcta.

37.3 Definiciones

- **Estado de pantalla**: combinacion de datos, permisos y contexto funcional que altera el comportamiento visible de una vista.
- **Modo solo lectura**: estado en el que la informacion es visible pero no editable por regla del sistema.

37.4 Reglas

- Cada pantalla debe declarar explicitamente estados de carga, vacio, error y restriccion cuando apliquen.
- Los estados visibles deben alinearse con estados reales del backend.
- Un cambio de estado funcional relevante debe reflejarse sin obligar al usuario a inferirlo.

37.4.1 Inventario minimo de pantallas

Pantalla	Acceso	Estados visibles minimos
Landing	publico	normal, vacio editorial, error basico de carga si consume datos
Registro	publico	editable, validacion, exito, error funcional, error tecnico
Login	publico	editable, validacion, error funcional, error tecnico
Recuperacion de acceso	publico	editable, confirmacion, error
Estado de activacion	autenticado	`PENDIENTE`, `ACTIVO`, `RECHAZADO`, `BLOQUEADO`, carga, error
Carga de comprobante	autenticado	editable, confirmacion, rechazo funcional, error tecnico
Calendario y picks	autenticado	carga, lista, vacio, editable, bloqueado, solo lectura, error
Predicciones especiales	autenticado	editable, bloqueado, solo lectura, error
Leaderboard	autenticado	carga, lista, vacio, error, estado con desglose si aplica
Perfil	autenticado	carga, datos visibles, vacio parcial, error
Admin usuarios y pagos	admin	carga, lista, vacio, accion permitida, accion restringida, error
Admin partidos y resultados	admin	carga, lista, edicion, confirmacion, error, resultado corregido
Admin scoring y auditoria	admin	carga, lista, confirmacion critica, error, sin permisos

37.4.2 Estados visibles especialmente sensibles

- `PENDIENTE`, `ACTIVO`, `RECHAZADO` y `BLOQUEADO` en activacion y contexto de cuenta.
- Partido editable o bloqueado en captura de picks.
- `SIN_PICK` como ausencia valida de prediccion guardada.
- Resultado corregido cuando el sistema deba comunicar una actualizacion relevante.
- Accion administrativa restringida cuando falten permisos o contexto funcional valido.

37.4.3 Reglas para estados de error

- Un error funcional debe indicar que regla impidio la accion.
- Un error tecnico debe permitir distinguir que la falla no fue del usuario cuando eso sea cierto.
- Un error no debe dejar la pantalla en un estado ambiguo sobre si la accion se aplico o no.

37.4.4 Reglas para estados de carga y vacio

- La carga debe sugerir que dato se esta esperando cuando eso mejore comprension.
- Un estado vacio debe orientar al siguiente paso util y no parecer una falla silenciosa.
- El vacio de picks no debe confundirse con `SIN_PICK` ya consolidado tras el lock.

37.5 Edge Cases

- Una pantalla puede pasar de editable a solo lectura entre carga inicial y envio de accion por un lock reciente; la vista debe reconciliarse correctamente.
- Un leaderboard puede requerir refresco visible cuando un resultado corregido cambie posiciones.
- Una pantalla administrativa puede tener acceso valido al modulo pero restricciones a una accion puntual segun permiso o contexto.

37.6 Invariantes

- El frontend no muestra un estado funcional que el backend no pueda sostener.
- Toda pantalla critica tiene un estado visible para exito, restriccion y error.
- Los estados visibles reducen ambiguedad operativa y no la aumentan.

37.7 Preguntas abiertas

- Confirmar si el detalle de partido existira como pantalla propia o si toda la experiencia de picks se resolvera desde una unica superficie de calendario.
- Confirmar en que superficies exactas vivira el aviso de resultado corregido.

37.8 Decisiones pendientes

- `Decision pendiente` `Bloquea frontend`: cerrar las superficies exactas donde se muestra el aviso de resultado corregido al participante.
- `Decision pendiente` `No bloqueante`: definir si el leaderboard por fase o jornada se representa como pantalla propia o como variante filtrada de una unica vista.

38 Accesibilidad y microcopy

38.1 Descripcion

Este documento define lineamientos funcionales de accesibilidad y de microcopy para Quiniela. Su objetivo es asegurar que el producto comunique acciones, restricciones, errores y estados con lenguaje claro, accesible y consistente con el comportamiento real del sistema.

38.2 Alcance

Aplica a textos de botones, mensajes de estado, validaciones, errores, confirmaciones y accesibilidad basica de la interfaz. No define una guia editorial completa de marca.

38.3 Definiciones

- **Microcopy**: texto corto orientado a guiar acciones, estados o errores.
- **Accesibilidad basica**: minimo de buenas practicas para que la interfaz sea comprensible y operable por una audiencia amplia.

38.4 Reglas

- El lenguaje debe ser simple, directo y no tecnico para el participante.
- Los mensajes del panel administrativo pueden ser mas precisos, pero no ambiguos ni cripticos.
- La accesibilidad no puede depender solo del color.
- Ningun mensaje debe ocultar una regla real del sistema o sugerir una accion inexistente.

38.4.1 Principios de microcopy

- describir primero el estado o problema y luego la accion siguiente;
- evitar tecnicismos cuando el usuario no los necesita;
- usar la misma palabra para el mismo concepto en todas las pantallas;
- no prometer exito hasta que la operacion haya sido confirmada.

38.4.2 Mensajes funcionales prioritarios

- cuenta pendiente de activacion;
- comprobante recibido;
- activacion aprobada o rechazada;
- pick guardado correctamente;
- pick bloqueado por cierre;
- partido sin pick al llegar el lock;
- prediccion especial bloqueada por cierre;
- resultado corregido y puntajes recalculados;
- accion administrativa realizada o rechazada.

38.4.3 Lineamientos de accesibilidad minima

- los estados importantes deben apoyarse en texto y no solo en color o icono;
- los CTA deben tener etiquetas claras y especificas;
- los errores de formulario deben aparecer cerca del campo o accion afectada;
- la jerarquia visual debe permitir identificar rapidamente titulo, accion principal y estado relevante;
- las tablas o rankings deben seguir siendo interpretables cuando cambie el orden o exista recalculo.

38.4.4 Tono recomendado

- claro;
- respetuoso;
- orientado a accion;
- transparente cuando el sistema corrige o restringe algo.

38.4.5 Mensajes que deben evitarse

- mensajes vagos como “ocurrio un problema” sin contexto util;
- mensajes que culpen al usuario cuando el fallo es tecnico;
- textos que sugieran que un pick fue guardado si la operacion aun no se confirmo;
- mensajes administrativos que oculten impacto real de una accion critica.

38.5 Edge Cases

- Un error tecnico puede requerir lenguaje diferente al error funcional aunque aparezca en la misma pantalla.
- El aviso de resultado corregido debe ser claro, pero no necesita exponer por defecto detalle profundo de auditoria.
- Un estado de cuenta bloqueada requiere tono firme y claro sin dejar ambiguedad sobre la restriccion.

38.6 Invariantes

- Los mensajes no ocultan reglas reales del sistema.
- La accesibilidad no depende solo del color.
- El microcopy mantiene consistencia terminologica con la documentacion del dominio.

38.7 Preguntas abiertas

- Confirmar si el proyecto quiere un glosario corto de microcopy prohibido y recomendado para mantener consistencia entre frontend y operacion.

38.8 Decisiones pendientes

- `Decision pendiente No bloqueante`: definir un catalogo operativo de mensajes clave por feature si el proyecto quiere controlar microcopy con mayor detalle.
- `Decision pendiente No bloqueante`: confirmar el tono institucional final para mensajes administrativos y comunicaciones visibles al participante.

39 Roadmap y fases

39.1 Descripcion

Este documento define la secuencia de fases del trabajo documental y su relacion con el desarrollo del proyecto. Su objetivo es ordenar el avance del repositorio por reduccion real de ambiguedad y no por un simple orden superficial de carpetas.

39.2 Alcance

Aplica al repositorio documental y a su uso como base de trabajo para backend y frontend.

39.3 Definiciones

- **Fase**: agrupacion de entregables con un objetivo comun.
- **Cierre funcional**: momento en que una seccion ya es suficientemente consistente para servir como referencia de trabajo.

39.4 Reglas

- El trabajo documental se prioriza por reduccion de ambiguedad.
- Dominio y datos se cierran antes del detalle tecnico fino.
- Ninguna fase debe avanzar ignorando contradicciones abiertas de la fase anterior.

39.4.1 Fases del roadmap documental

Fase 1: nucleo del dominio

- objetivo: cerrar reglas de producto, estados, picks, locks, scoring, desempates y Edge Cases;
- entregables principales: `00_Overview`, `01_Producto_y_Reglas`, partes nucleares de `05_Datos`;
- resultado esperado: lenguaje comun y reglas del juego sin ambiguedad fuerte.

Fase 2: arquitectura y aterrizaje tecnico

- objetivo: traducir el dominio ya cerrado a arquitectura, modulos, fronteras y contratos conceptuales;
- entregables principales:
  - `02_Arquitectura`;

  - `03_Backend`;
  - `04_Frontend`;
  - `06_Integraciones`;
  - `07_Seguridad_y_Operacion`.
- resultado esperado: base tecnica coherente para construir ambos repositorios de implementacion.

Fase 3: calidad, experiencia y gestion

- objetivo: cerrar testing, UI/UX, backlog documental, riesgos y ciclo de aceptacion;
- entregables principales: `08_Testing`, `09_UI_UX`, `10_Planeacion`;
- resultado esperado: documentacion lista para trabajo iterativo, revision y salida controlada.

39.4.2 Estado actual del roadmap

Al momento de esta iteracion:

- la Fase 1 esta sustancialmente cubierta;
- la Fase 2 esta mayormente cubierta en arquitectura, backend, frontend, integraciones y operacion;
- la Fase 3 esta en consolidacion documental y editorial.

39.4.3 Siguiente tramo recomendado

- pasada de consistencia cruzada entre secciones;
- limpieza editorial y de maquetacion;
- compilacion del PDF maestro;
- refinamiento puntual de decisiones pendientes que aun bloquean frontend.

39.5 Edge Cases

- Una fase puede requerir retrocesos puntuales si una decision abierta descubierta tarde afecta documentos ya avanzados.
- Un cierre parcial de fase es aceptable si los pendientes quedaron explicitamente registrados y no se presentan como decisiones ya resueltas.

39.6 Invariantes

- No se adelanta detalle tecnico fino cuando el dominio aun esta abierto.
- Ninguna fase se considera cerrada si deja contradicciones normativas sin registrar.
- El roadmap documental debe poder explicar por que una seccion se trabajo antes que otra.

39.7 Preguntas abiertas

- Confirmar si el proyecto quiere convertir estas fases en hitos formales por semana, sprint o solo mantenerlas como secuencia logica de trabajo.

39.8 Decisiones pendientes

- `Decision pendiente No bloqueante`: decidir si el roadmap documental se seguira manteniendo como documento vivo o si solo reflejara grandes cortes de avance.

40 Backlog documental

40.1 Descripcion

Este documento define el inventario ordenado de pendientes documentales del repositorio. Su objetivo es distinguir claramente trabajo faltante de decisiones abiertas, revisiones editoriales y tareas de consolidacion transversal.

40.2 Alcance

Aplica a contenido faltante, revisiones, consistencia cruzada, maquetacion y cierres de iteracion posteriores.

40.3 Definiciones

- **Backlog documental**: lista priorizada de trabajo pendiente sobre la documentacion.
- **Refinamiento**: mejora de claridad, consistencia o mantenibilidad sobre un documento ya existente.

40.4 Reglas

- Todo pendiente nuevo debe entrar al backlog con prioridad visible.
- El backlog no reemplaza `Decisiones pendientes`; la complementa.
- Un pendiente editorial no debe confundirse con una decision de producto aun abierta.

40.4.1 Backlog activo por prioridad

Prioridad alta

- ejecutar una pasada de consistencia cruzada entre:
  - `01_Producto_y_Reglas`;
  - `05_Datos`;
  - `02_Arquitectura`;
  - `03_Backend`;
  - `04_Frontend`.

- consolidar y limpiar preguntas abiertas ya resueltas en documentos de dominio y frontend;
- compilar y validar el PDF maestro del repositorio.

Prioridad media

- reducir warnings de maquetacion en tablas largas de backend, frontend, testing y UI/UX;
- decidir si algunos trees o tablas requieren convertirse en diagramas o anexos mas legibles;
- revisar uniformidad de redaccion entre secciones operativas y secciones normativas.

Prioridad baja

- evaluar si conviene una version resumida del reglamento para participantes;
- evaluar si conviene una plantilla mas rica para anexos o tablas extensas;
- evaluar si se agregan referencias cruzadas internas mas explicitas entre documentos.

40.4.2 Tipos de trabajo admitidos en backlog

- contenido faltante;
- refinamiento de claridad;
- consistencia cruzada;
- maquetacion;
- compilacion y validacion;
- reorganizacion documental justificada.

40.5 Edge Cases

- Un backlog puede crecer aunque la documentacion avance; eso es aceptable si los nuevos pendientes estan mejor delimitados y no mas ambiguos.
- Una decision cerrada puede generar tareas nuevas de backlog para propagarla a varias secciones.

40.6 Invariantes

- Ningun trabajo documental relevante queda fuera del backlog o del changelog.
- Un pendiente de contenido no debe esconderse como simple tarea editorial.
- El backlog debe poder leerse sin revisar todo el historial conversacional.

40.7 Preguntas abiertas

- Confirmar si el backlog se quiere clasificar en el futuro por prioridad, dependencia y estado de ejecucion formal.

40.8 Decisiones pendientes

- `Decision pendiente` `No bloqueante`: decidir si el backlog documental mantendra estados formales como pendiente, en curso y cerrado o si bastara con prioridad y changelog.

41 Decisiones pendientes

41.1 Descripcion

Este documento concentra las decisiones aun abiertas del proyecto en un registro unico, visible y trazable. Su funcion es evitar que reglas importantes queden dispersas en conversaciones o notas informales.

41.2 Alcance

Aplica a reglas de negocio, arquitectura, datos, integraciones y criterios operativos que todavia no han sido cerrados formalmente.

41.3 Definiciones

- **Decision pendiente**: definicion necesaria aun no cerrada formalmente.
- **Bloqueante**: decision cuya ausencia impide documentar o construir correctamente una parte relevante del sistema.
- **Riesgo**: decision abierta que no bloquea por completo, pero puede introducir retrabajo, inconsistencia o deuda de producto.
- **Criticidad**: alcance del impacto de una decision abierta sobre frontend, backend o ambos.

41.4 Reglas

- Toda decision pendiente debe clasificarse por tipo y criticidad.
- Una decision cerrada debe actualizar su documento fuente y el changelog.
- La version mas reciente aprobada en este repositorio prevalece sobre notas previas y conversaciones no integradas al registro.
- Este registro no reemplaza al documento fuente de cada regla; solo centraliza el estado de las definiciones abiertas.

41.5 Registro activo

41.5.1 Prioridad alta

1. **Politica de visibilidad de Campeon y Goleador frente a otros usuarios**
   Tipo: `Decision pendiente`
   Criticidad: `Bloquea frontend`
   Documento responsable: `01_03_Picks_y_Locks`
   Motivo: falta definir si estas predicciones se ocultan siempre, se publican al cierre del primer partido o se revelan en otro momento controlado.

2. **Superficies y duracion del aviso de resultado corregido**
   Tipo: `Decision pendiente`
   Criticidad: `Bloquea frontend`
   Documento responsable: `01_05_Casos_Borde`
   Motivo: falta definir en que vistas aparece el aviso al usuario y durante cuanto tiempo debe mantenerse visible.

3. **Presentacion visual del desglose de puntos**
   Tipo: `Decision pendiente`
   Criticidad: `Bloquea frontend`
   Documento responsable: `01_04_Scoring_y_Desempates`
   Motivo: falta definir donde se muestra el aporte de picks por partido y predicciones especiales sin alterar el ranking oficial.

41.5.2 Prioridad media

4. **Precision visual de puntajes no enteros**
   Tipo: `Decision pendiente`
   Criticidad: `Bloquea frontend`
   Documento responsable: `01_04_Scoring_y_Desempates`
   Motivo: la logica interna ya conserva precision exacta, pero falta fijar con cuantos decimales se mostrara el puntaje al usuario.

5. **Presentacion del flujo separado de predicciones especiales**
   Tipo: `Decision pendiente`
   Criticidad: `Bloquea frontend`
   Documento responsable: `01_03_Picks_y_Locks`
   Motivo: la separacion funcional ya esta cerrada, pero falta bajar la decision a comportamiento de interfaz y navegacion.

6. **Tratamiento visual de predicciones especiales en leaderboard por jornada**
   Tipo: `Riesgo`
   Criticidad: `No bloqueante`
   Documento responsable: `01_04_Scoring_y_Desempates`
   Motivo: falta confirmar si la vista por jornada excluye visualmente los puntos de Campeon y Goleador aunque el total general los integre.

41.5.3 Prioridad baja

7. **Persistencia explicita de `SIN_PICK`**
   Tipo: `Decision pendiente`
   Criticidad: `No bloqueante`
   Documento responsable: `01_03_Picks_y_Locks` y `05_Datos`
   Motivo: la regla funcional ya esta cerrada, pero falta decidir si se persiste como estado explicito o si solo se deriva del negocio.

8. **Distincion visual entre correccion por API y override manual**
   Tipo: `Riesgo`
   Criticidad: `No bloqueante`
   Documento responsable: `01_05_Casos_Borde`

Motivo: la prioridad operativa ya esta definida, pero falta decidir si la interfaz hace visible el origen de la correccion.

9. **Version resumida del reglamento para participantes**
   Tipo: `Riesgo`
   Criticidad: `No bloqueante`
   Documento responsable: `01_01_Reglamento_V1`
   Motivo: el reglamento actual sirve como fuente normativa, pero aun no se decide si se publicara una version mas breve para consumo general.

41.6 Edge Cases

- Una decision puede bajar de criticidad si el documento fuente ya define un comportamiento por defecto suficiente para desarrollo interno.
- Una decision cerrada en conversacion no se considera resuelta hasta que quede reflejada en su documento fuente y en este registro.

41.7 Invariantes

- No debe haber decisiones relevantes abiertas solo en conversaciones informales.
- Toda decision abierta de producto debe tener un documento responsable identificable.
- La lista priorizada debe poder leerse sin necesidad de revisar todos los documentos fuente.

41.8 Preguntas abiertas

- No hay preguntas abiertas sobre el formato del registro en esta fase; el siguiente paso es mantenerlo sincronizado conforme avancemos documento por documento.

42 Riesgos y mitigaciones

42.1 Descripcion

Este documento define el catalogo de riesgos funcionales, operativos y documentales del proyecto, junto con una mitigacion propuesta. Su objetivo es tratar la incertidumbre del producto y de la documentacion como parte real de la planeacion.

42.2 Alcance

Aplica a riesgos que afecten construccion, operacion, claridad normativa, integraciones y sostenibilidad documental.

42.3 Definiciones

- **Riesgo**: evento potencial con impacto negativo sobre producto, operacion o desarrollo.
- **Mitigacion**: accion propuesta para reducir probabilidad, impacto o retrabajo.

42.4 Reglas

- Todo riesgo relevante debe tener mitigacion propuesta o aceptacion explicita.
- Los riesgos documentales se tratan como riesgos reales del proyecto.
- Un riesgo repetido no debe reaparecer sin trazabilidad de su estado anterior.

42.4.1 Catalogo minimo de riesgos

Riesgo	Impacto relativo	Mitigacion propuesta
reglas abiertas sobre visibilidad de predicciones especiales	alto	cerrar decision en documento responsable y propagar a frontend, UI/UX y backlog
ambiguedad sobre superficies del aviso de resultado corregido	alto	resolver la decision en dominio y reflejarla en frontend y microcopy
desalineacion entre frontend, backend y dominio	alto	ejecutar pasada de consistencia cruzada antes de cierre de iteracion
dependencia de proveedor externo aun no seleccionado	alto	mantener integracion en nivel conceptual y documentar contrato minimo esperado
maquetacion deficiente en tablas largas	medio	refinar tablas, dividir contenido o mover detalle a anexos si se vuelve necesario
sobredocumentacion o duplicacion de reglas	medio	conservar documento responsable por tema y eliminar duplicados en pasada editorial
decision tardia sobre roles admin y super admin	medio	mantener diferencia documentada como pendiente no bloqueante hasta cierre funcional real
documentacion desactualizada respecto a decisiones nuevas	alto	actualizar documento fuente, registro de decisiones y changelog en el mismo ciclo de cambio

42.4.2 Riesgos documentales especificos

- que un pendiente siga vivo en conversacion pero no en su documento fuente;
- que una seccion tecnica asuma decisiones aun no cerradas en producto;
- que la modularidad del repo se pierda por mover demasiadas reglas a un solo archivo.

42.4.3 Riesgos operativos especificos

- correccion manual de resultado sin suficiente contexto;
- recalculo que no deje trazabilidad entendible;
- acciones administrativas criticas con poca claridad en UI.

42.5 Edge Cases

- Un riesgo puede bajar de prioridad sin desaparecer si su mitigacion depende de una decision futura aun no cerrada.
- Un riesgo tecnico puede transformarse en riesgo documental si la documentacion no refleja correctamente la restriccion real del sistema.

42.6 Invariantes

- Todo riesgo relevante tiene una mitigacion propuesta o una aceptacion explicita.
- Los riesgos documentales no se consideran de menor importancia por no vivir en codigo.
- El registro de riesgos debe poder usarse para priorizar refinamientos del repositorio documental.

42.7 Preguntas abiertas

- Confirmar si el proyecto quiere formalizar una matriz adicional de probabilidad e impacto o si este catalogo cualitativo es suficiente para v1.

42.8 Decisiones pendientes

- `Decision pendiente` `No bloqueante`: decidir si el registro de riesgos se mantendra como lista cualitativa o migrara a una matriz mas formal en iteraciones futuras.